

Site Automation

Operations runbooks: health checks, troubleshooting,
ServiceNow, POA&M and patching - with AAP and, later, ACT

Setup guide for new AAP admins | site-automation 0.4.0 | September 2026

What this is. A step-by-step guide to setting up the site's operations runbooks in Ansible Automation Platform: health checks (system, compliance, MariaDB, certificates, patching), troubleshooting, ServiceNow tickets, POA&M status and patching, and later ACT (GenAI). It starts with what each AAP object *is* and walks through every click, saying what each step does, why, and what you should see.

How to read it

New to Ansible or AAP: read chapter 1 first, then do chapter 2 in order. Inventory already in AAP? Chapter 3 replaces the inventory parts of chapter 2. Chapters 4 to 6 are reference: open them when you build a workflow, read a finding, or add ACT.

Rendered from [docs/](#) in the site-automation repository (version 0.4.0). The repository is the source of truth.

Contents

1. Start here: the pieces and how they fit	5
What the runbooks do	5
The ideas behind Ansible, in plain words	5
The AAP objects, and what each one is for	6
How a health check works, start to finish	7
The findings: the one idea that makes everything modular	7
Where things are	8
The order to do things in	8
2. Setting up AAP, step by step	9
Before you start	9
Step 1. Put the repository in your work Git, with your inventory	9
Step 2. Credential types	10
Step 3. Credentials	10
Step 4. Project	11
Step 5. Inventory (filled from the repository)	11
Step 6. The first job template: Health check	12
Step 7. Run it on one host and read the output	12
Step 8. Tune the thresholds	13
Step 9. The other job templates	13
Step 10. ServiceNow	16
Step 11. Workflows and schedules	16
Step 12. Who can do what	16
If something goes wrong	17
3. Using the inventory you already have in AAP	18
The easy way: your settings files, already named after your groups	18
How variables work in AAP (two minutes)	19
Step 1. Look at what you have	20
Step 2. Create the groups the runbooks use	20
Step 3. Settings for every host (inventory variables)	21
Step 4. Settings for particular groups	22
Step 5. Settings for one host (only when one host is different)	22
Step 6. Use your inventory in the job templates	23

Step 7. Check that it worked	23
Which YAML files in the repository do I change?	23
4. Workflows and schedules	24
How to build any workflow, click by click	24
Linux workflows	25
Windows workflows	31
Linux and Windows together	32
Schedules	32
Things that trip people up	33
5. Dry runs: try anything, change nothing	35
Where to set it	35
What each template does in a dry run	36
Examples	36
What a dry run cannot tell you	37
6. The runbooks: every check and what to do	38
Health check (playbooks/health_check.yml)	38
Troubleshoot (playbooks/troubleshoot.yml)	45
Certificate report (playbooks/cert_report.yml)	46
POA&M status (playbooks/poam_status.yml)	46
ServiceNow tickets (playbooks/servicenow_tickets.yml)	46
ServiceNow health (playbooks/servicenow_health.yml)	47
Patch hosts (playbooks/patch_hosts.yml)	47
Apply approved ACT fix (playbooks/act_fix_approved.yml)	47
Not built yet (and why)	48
7. Adding ACT (GenAI) later	49
How they fit: checks find WHAT, ACT finds WHY	49
The three levels	49
Turn it on	50
Write your own check (it works with the report, tickets and ACT automatically)	52
Use ACT in a playbook of your own	53
What ACT is told, and what it cannot do	54
8. Windows servers	55
Step 1. WinRM over HTTPS on the servers	56

Step 2. The execution environment56

Step 3. The credential57

Step 4. The Windows inventory57

Step 5. Test the connection57

Step 6. The job templates58

Step 7. Workflows and schedules59

Start small: the pilot order59

Patching through Software Center59

ACT on the servers (C:\ProgramData\act)61

Settings: what each check looks at, and how to change it61

If something goes wrong63

Not in this release (next)63

Appendix: credential types to paste into AAP64

ServiceNow API (servicenow_api.yml)64

MariaDB monitor (mariadb_monitor.yml)64

Keystore password (keystore_password.yml)65

STIG Manager API (stigman_api.yml)65

ACT model key (act_model_key.yml)65

ACT GenAI beta key (act_genai_beta_key.yml)66

1. Start here: the pieces and how they fit

Confused by all the moving pieces?

Read HOW_IT_FITS_TOGETHER.md (or its PDF in [docs/pdf/](#)). It shows what the code, the inventory, the credentials and the survey each do, how a setting travels from a YAML file to a host, and how to update the repository safely.

This repository holds **runbooks**: jobs that check Linux servers and help troubleshoot them. You run them from **Ansible Automation Platform (AAP)**, by hand or on a schedule. This page explains the few ideas you need. SETUP_AAP.md then walks you through the setup, one click at a time.

What the runbooks do

Runbook	What it answers	When it runs
Health check	Is anything wrong on these servers right now? Disk, mounts, services, load/memory, time, network, logging, SELinux, fapolicyd, auditd, accounts, certificates, patching, MariaDB	Daily (the "daily" set) and weekly (the "weekly" set)
Troubleshoot	Someone reports a problem on a host. What does it look like?	On demand
Certificate report	Which certificates expire soon, on every server, in one list?	Weekly
POA&M status	Which POA&M items are overdue or due soon? Which open CAT I/II findings have no POA&M item?	Weekly
ServiceNow tickets	Opens or updates a ticket for each problem a check found. Notes the ones that cleared	After every check (as a workflow step)
ServiceNow health	Is ServiceNow answering? Are the MID Servers up?	Hourly
Patch hosts	Installs updates, and proves every service came back. Reboots only with <code>automatic_restarts: yes</code>	Monthly, after an approval
Apply approved ACT fix	Applies exactly the fix a person approved	Only inside a workflow, after an approval

Every check is **read-only**. It looks and reports, and changes nothing. Patching is the only runbook that changes servers, and it waits for an approval first.

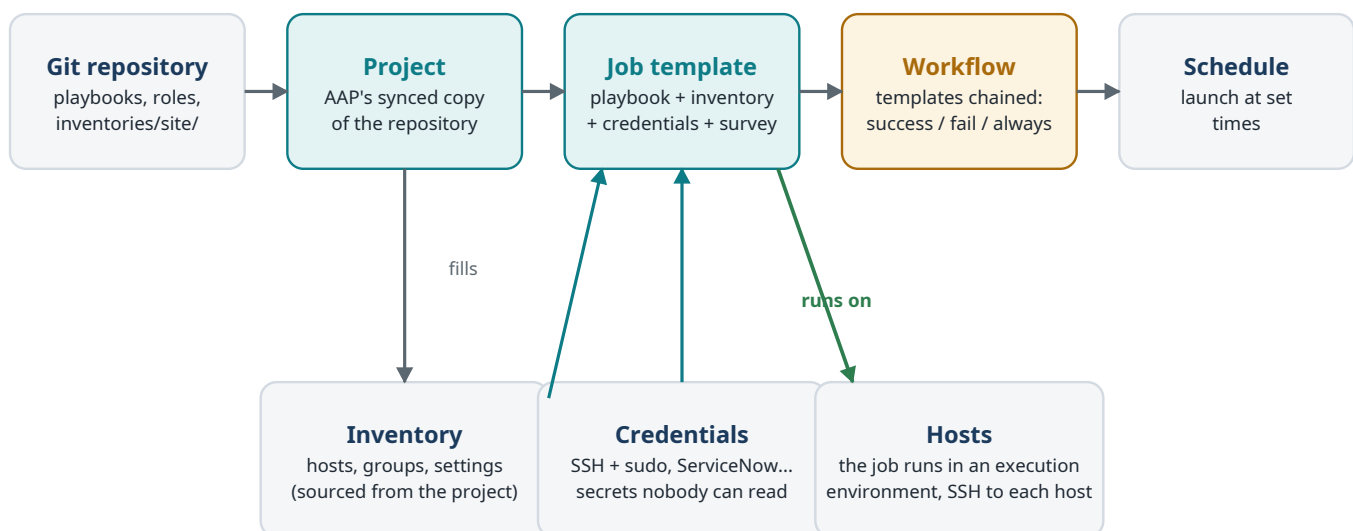
The ideas behind Ansible, in plain words

- **Host**: a server Ansible connects to, over SSH, the way you would.
- **Inventory**: the list of hosts, sorted into **groups** such as `stigman` (the STIG Manager servers) or `mariadb`. The built-in group `all` holds every host. A group can also carry **settings (variables)**, for example "on the database hosts, the MariaDB container is called servicenow-mariadb".

- **Task:** one step, such as "run `df` and read the result". A **module** is the tool a task uses (`command`, `shell`, `uri`, `set_fact`...).
- **Role:** a folder of tasks that does one job, plus its settings. `roles/check_disk/` is "check the disks". Its settings, with their defaults, are in `roles/check_disk/defaults/main.yml`.
- **Playbook:** the file you run. It says which hosts, and which roles or tasks to run on them. `playbooks/health_check.yml` says "on every host, run the checks I was given".
- **become:** run as root through `sudo`. The checks need it to read root-only information (the audit log, `/etc/shadow`).
- **Check mode:** a dry run. Ansible shows what it would change and changes nothing. Every runbook here works in check mode, and the checks give the same answers in both modes.
- **Idempotent:** running it twice does no harm. The second run changes nothing.

The AAP objects, and what each one is for

AAP is a web front end that runs playbooks for you. It keeps the passwords, records every run, and runs things on a schedule. You set up a handful of objects once. Each one points at the next:



The AAP objects you create once. The job template is what you launch; a workflow chains templates; a schedule launches either.

AAP object	What it is	Here
Project	A copy of a Git repository that AAP keeps in sync	<code>site-automation</code> (this repository)
Inventory	The hosts and groups, and their settings	<code>Linux servers</code> , filled from <code>inventories/site/</code> in this repository
Credential type	A form for a kind of secret, and how to hand it to the job	<code>ServiceNow API</code> , <code>MariaDB monitor</code> , <code>ACT model key...</code> (<code>aap/credential_types/</code>)
Credential	One filled-in form: the actual secret. Nobody can read it back	<code>Linux ssh (sudo)</code> , <code>ServiceNow API...</code>

AAP object	What it is	Here
Execution environment	The container a job runs in (Ansible, Python, tools)	The Minimal execution environment is enough. This repository needs no extra collections
Job template	"Run this playbook, on this inventory, with these credentials" plus the questions to ask (survey). This is what you launch	Health check , Troubleshoot , Patch hosts ...
Workflow	Job templates chained together. The next step can depend on the last one's result (success, failure, always). It can stop for an approval	Daily health = Health check, then ServiceNow tickets
Schedule	"Launch this template or workflow at these times"	Daily health every day at 06:00

Three more words you will see:

- **Survey:** the questions a template asks when you launch it, such as "which checks?". Each answer becomes a variable for the playbook.
- **Limit:** run on only some hosts or groups, for example `snowdb01.example.mil` or `mariadb_hosts`.
- **Artifacts:** what a job hands to the next step of a workflow. The health check hands its findings to the ServiceNow step this way (`set_stats` in the playbook).

How a health check works, start to finish

1. You click **Launch** on the *Health check* template (or its schedule fires).
2. AAP syncs the project, starts the execution environment, and connects to each host over SSH with the Machine credential.
3. On each host, each chosen check runs read-only commands and turns what it sees into **findings**. One finding is one thing that is wrong, in plain words. It includes a command you can paste to look further:

```
[CRITICAL] disk: /var is 96% full (critical at 95%), 1.2 GiB free of 40.0 GiB
look: timeout 60 du -xh --max-depth=2 /var 2>/dev/null | sort -h | tail -15
```

4. The job prints a summary per host and for the whole fleet. It then **fails the hosts that have findings**. The red status is on purpose: it is how a workflow knows it should open tickets (or ask ACT, or ask for approval).
5. In the *Daily health* workflow, the *ServiceNow tickets* step runs next. It opens a ticket for each finding, or updates the ticket that is already open.

The findings: the one idea that makes everything modular

Every check produces the same kind of finding:

Field	Example	Used by
<code>check</code>	<code>disk</code>	reports, tickets
<code>id</code>	<code>disk:/var</code>	ServiceNow updates the same ticket every run, instead of opening a new one
<code>severity</code>	<code>critical</code> or <code>warning</code>	pass/fail, ticket urgency

Field	Example	Used by
summary	<code>/var is 96% full (critical at 95%)</code>	people
hint	<code>du -xh --max-depth=2 /var</code> ...	people, and ACT, which runs it to collect evidence

The report, the ServiceNow tickets and ACT only ever read findings. That is why:

- a new check you write shows up in reports and tickets with no other change;
- ACT works on top of **any** check. Add `use_act=true` and ACT reads the findings and the hint commands' output, then explains the root cause (ADDING_ACT.md).

Where things are

Path	What
<code>playbooks/</code>	the files AAP runs (one per job template)
<code>playbooks/group_vars/</code>	your settings , one file per AAP group: edit these (USING_YOUR_AAP_INVENTORY.md)
<code>roles/check_*/</code>	one folder per check. Its settings are in <code>defaults/main.yml</code>
<code>roles/site_findings/</code>	starts, reports and publishes findings, and decides pass/fail
<code>roles/servicenow/</code> , <code>roles/troubleshoot/</code> , <code>roles/patch/</code> , <code>roles/poam/</code>	the other runbooks
<code>roles/site_act/</code>	the bridge to ACT (GenAI)
<code>inventories/example/</code>	a sample inventory and settings. Copy it to <code>inventories/site/</code>
<code>aap/credential_types/</code>	the credential types to create in AAP
<code>poam/poam.csv</code>	your POA&M list (for the POA&M runbook)
<code>docs/</code>	this guide, SETUP_AAP.md, USING_YOUR_AAP_INVENTORY.md, RUNBOOKS.md, WORKFLOWS_AND_SCHEDULES.md, ADDING_ACT.md; PDFs in <code>docs/pdf/</code>

The order to do things in

1. **Set up and run one health check by hand** (SETUP_AAP.md, steps 1 to 8). Your inventory is already in AAP? Use USING_YOUR_AAP_INVENTORY.md for the inventory part. Read its output. Adjust thresholds in the inventory until the findings are the ones you care about.
2. **Add the other job templates** (step 9) and try each once.
3. **Add ServiceNow** (step 10), and build the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md).
4. **Schedule** the workflows.
5. **Later: add ACT** to the runbooks you trust (ADDING_ACT.md).

2. Setting up AAP, step by step

Every step says **what** you are doing, **why**, the **clicks**, and what you should **see** when it worked. Menu names are from AAP 2.7 (they are the same in 2.5 and 2.6). If you have not yet, read `START_HERE.md` first. It explains the words used here.

Before you start

You need:

- An AAP account that can create projects, inventories, credentials and templates in your organization (an **Organization admin** does it all).
- A **Linux account on the servers** that AAP logs in as, for example `svc_aap`. It needs an SSH key and sudo. With `NOPASSWD` sudo, add it to `check_accounts_nopasswd_allowed`, or the accounts check will report it.
- A **Git repository at work** that AAP can reach (GitLab, Bitbucket, GitHub Enterprise).
- For ServiceNow: an **API account** (step 10).

Step 1. Put the repository in your work Git, with your inventory

Already have your inventory in AAP?

Then you do not need `inventories/site/`. Push the repository without it (skip items 1-3 below), skip step 5, and follow `USING_YOUR_AAP_INVENTORY.md` for the groups and settings to add.

What. Copy this repository into your work Git, and create your real inventory in it. (Already pushed an earlier release at work? Update it with `scripts/update-from-release.ps1` in VS Code, as in `HOW_IT_FITS_TOGETHER.md`, "Updating the repository at work". Then skip to step 2.)

Why. AAP reads playbooks from Git, never from your laptop. The inventory lives in the same repository, so host lists and thresholds are versioned and reviewed like code. Every change has a who, a when and a why.

1. Extract the release zip (right-click › **Extract All**). In the extracted folder, copy the folder `inventories\example` and name the copy `inventories\site`.
2. Edit `inventories/site/hosts.yml`. Replace the example host names with yours and keep the group names: the playbooks use them. A host can be in several groups.
3. Edit `inventories/site/group_vars/all.yml`. These settings apply to every host: the LDAP and SIEM servers each host must reach, the account AAP logs in as (`svc_aap`), and the thresholds. Each setting is explained in `roles/check_<name>/defaults/main.yml`.
4. Put it in your work Git, with VS Code:
 - Create an empty repository on your Git server (for example `site-automation`) and copy its clone URL.
 - In VS Code: **Source Control** › **Clone Repository** › paste the URL › pick a folder, such as `C:\git\site-automation`.
 - Copy everything from the extracted release folder into that folder.
 - In **Source Control**: type a message, click **Commit** (answer Yes to "stage all changes"), then **Sync Changes / Publish Branch**.

The same from a command line: `git add -A, git commit -m "site-automation 0.3.1", then git push`.

- Note the two values AAP needs in step 4: the **clone URL** (the one you pasted above) and the **branch** (shown at the bottom left of VS Code, usually `main`). AAP does not read them from your copy; you type them in once.

You should see the files in your Git server's web page, including `inventories/site/`.

Step 2. Credential types

What. Teach AAP the kinds of secrets these runbooks use, one form per kind.

Why. AAP ships with types for SSH (Machine) and Git (Source Control). The ServiceNow login and the others are custom. A credential type says which fields the form has and how the secret reaches the job. Every type here passes it as an **environment variable** in the execution environment. The playbook must ask for it on purpose, and it never shows in job output or Git.

For each file in `aap/credential_types/` that you need now:

- Automation Execution > Infrastructure > Credential Types > Create credential type.**
- Name:** the name at the top of the file (for example `ServiceNow API`).
- Input configuration:** paste the `inputs:` part of the file, *without* the `inputs:` line.
- Injector configuration:** paste the `injectors:` part, *without* the `injectors:` line.
- Click **Create credential type**.

File	Name	Needed for
<code>servicenow_api.yml</code>	ServiceNow API	tickets, ServiceNow health (step 10)
<code>mariadb_monitor.yml</code>	MariaDB monitor	the MariaDB check on a containerized database (root login refused there); optional on host installs
<code>keystore_password.yml</code>	Keystore password	optional: certs check on a PKCS12 keystore
<code>stigman_api.yml</code>	STIG Manager API	optional: POA&M cross-check with STIG Manager
<code>act_model_key.yml</code>	ACT model key	later, for ACT (ADDING_ACT.md): GenAI.mil and Ask Sage keys
<code>act_genai_beta_key.yml</code>	ACT GenAI beta key	only for ACT with the GenAI.mil beta provider

You should see each new type in the Credential Types list.

Step 3. Credentials

What. Store the actual secrets, once.

Why. This is the only place a password or key is kept. Templates point at the credential by name. Nobody, including you, can read the secret back, and it is never in Git or job output.

Automation Execution > Infrastructure > Credentials > Create credential:

Name	Credential type	Fill in
Linux ssh (sudo)	Machine	Username <code>svc_aap</code> , SSH Private Key , Privilege Escalation Method <code>sudo</code> (plus the sudo password if sudo asks for one)
site-automation git	Source Control	only if the repository is private: user name and token, or an SSH key
ServiceNow API	ServiceNow API	instance URL, API user, password (step 10)
MariaDB monitor	MariaDB monitor	needed when MariaDB runs in a container (the ServiceNow database); optional otherwise. See RUNBOOKS.md for the account to ask the DBA for

You should see the credentials listed, with secret fields shown as **ENCRYPTED**.

Step 4. Project

What. Tell AAP where the repository is.

Why. A project is AAP's copy of the repository. Templates pick their playbook from it.

1. **Automation Execution > Projects > Create project.**
2. **Name** `site-automation`. **Organization:** yours.
3. **Execution environment:** **Minimal execution environment** (`ee-minimal`). These runbooks use only modules that come with Ansible itself, so no extra collections or custom execution environment are needed (useful on a disconnected network).
4. **Source control type** `Git`. **Source control URL:** from step 1. **Source control branch/tag/commit:** `main`. Later, pin a release tag such as `v0.3.0`. **Source control credential:** `site-automation git` if the repository is private.
5. Tick **Update revision on launch**. Every job then syncs first, so a change you push is used right away.
6. Click **Create project**.

You should see the project's **Last job status** turn **Successful** after a few seconds. If it fails, open the sync job: it is almost always the URL, the branch name, or the credential.

Step 5. Inventory (filled from the repository)

(Skip this step if your inventory is already in AAP: see `USING_YOUR_AAP_INVENTORY.md`.)

What. Create the `Linux servers` inventory and fill it from `inventories/site/hosts.yml`.

Why. Sourcing the inventory from the project brings in hosts, groups and the settings in `group_vars/`, all at once, and keeps them in step with Git. You do not type hosts into AAP by hand.

1. **Automation Execution > Infrastructure > Inventories > Create inventory > Create inventory.** **Name** `Linux servers`. Click **Create inventory**.
2. Open it > **Sources** tab > **Create source:**
 - **Name** `site inventory from Git`
 - **Source** `Sourced from a Project`
 - **Project** `site-automation`
 - **Inventory file** `inventories/site/hosts.yml`

- Tick **Overwrite**, **Overwrite variables** and **Update on launch**.

3. Click **Create source**, then **Launch inventory update**. The rocket icon also starts it.

You should see, under **Groups**, `stigman`, `mariadb_hosts`, `patch_hosts`... Under **Hosts** you should see your servers. Open a group: its **Variables** are the ones from `group_vars/<group>.yaml`.

Step 6. The first job template: Health check

What. A template that runs `playbooks/health_check.yaml`.

Why. A job template is the thing you launch. It fixes which playbook, which inventory and which credentials. Its survey asks which checks to run.

1. **Automation Execution > Templates > Create template > Create job template.**
2. Fill in:

Field	Value	Why
Name	Health check	
Job type	Run	(Check also works: the checks read the same way)
Inventory	Linux servers	
Project	site-automation	
Playbook	playbooks/health_check.yaml	
Execution environment	Minimal execution environment	
Credentials	Linux ssh (sudo) (+ MariaDB monitor if you made it)	how AAP logs in
Limit	leave empty, tick Prompt on launch	so you can run it on one host
Privilege escalation	ticked	the checks read root-only files

3. Click **Create job template**.

4. **Survey** tab > **Create survey question**:

- **Question** Which checks?
- **Answer variable name** `health_checks`
- **Answer type** *Multiple Choice (multiple select)*
- **Choices**, one per line: `daily`, `weekly`, `all`, `disk`, `mounts`, `services`, `performance`, `time`, `network`, `logging`, `selinux`, `fapolicyd`, `auditd`, `accounts`, `certs`, `patching`, `mariadb`
- **Default answer** `daily`. Tick **Required**. Click **Create question**.

5. Turn the survey **on** (the switch at the top of the Survey tab).

You should see the template in the Templates list with a rocket icon.

Step 7. Run it on one host and read the output

What. Launch the health check on a single server.

Why. Start small. You will see what the checks find on a real host, and whether the thresholds suit you.

1. Click the rocket on **Health check**. **Limit**: one host name. Survey: **daily**. Click **Next**, then **Launch**.
2. The job output scrolls by. The parts to read:
 - **Check disk | run, Check services | run...**: each check running (green **ok** lines are normal).
 - **Report | result for this host**: the verdict in plain words. Either **HEALTHY**. **Checks run: ...** or a list of **[CRITICAL]** / **[WARNING]** findings. Each finding has a **look:** line: a command you can paste on the host to see more.
 - **Report | fleet summary**: one line for all hosts, for example **3 healthy, 1 with findings**.
 - A red **FAIL this host because it has findings** at the end. This is **on purpose**: a job with findings fails, so a workflow can react (open tickets). It does not mean the playbook broke.
3. The **Details** tab shows who launched it, when, and the exact revision of the repository.

You should see a finished job, green if the host is healthy, red with findings otherwise.

Step 8. Tune the thresholds

What. Make the findings match what you care about.

Why. Defaults are a starting point. If a warning is expected on some host, change the setting for that host or group, not the code.

1. Change the setting in **inventories/site/group_vars/<group>.yaml** (every host: **all.yaml**). A single host can have **inventories/site/host_vars/<host>.yaml**. Examples:

YAML

```
check_disk_overrides:
  /var/log/audit: {warn: 70, crit: 85}      # this filesystem: warn earlier
check_services_ignore_failed: ['^dnf-makecache']
site_fail_on: [critical]                   # warnings are reported but do not fail the job
```

2. Commit and push. In AAP, run the inventory source again (**Inventories > Linux servers > Sources > Launch inventory update**). A setting in **group_vars** reaches AAP only through the inventory source.
3. Launch the health check again.

You should see the finding gone, or at its new threshold.

Step 9. The other job templates

Create each the same way as step 6: **Inventory Linux servers**, **Project site-automation**, **Execution environment Minimal**. For the ones that log in to hosts, tick **Privilege escalation** and leave **Limit** empty with **Prompt on launch** ticked. Only the differences are listed. A survey question is *Multiple Choice (single select)* unless it says otherwise.

Three templates run on the controller, not on hosts:

POA&M status, ServiceNow tickets and ServiceNow health. For these, do **not** tick Prompt on launch for Limit. A workflow's Limit (for example **patch_hosts**) is passed to every step that prompts for one. These playbooks run on **localhost**, so the Limit would match nothing and the step would silently do nothing, and still show green.

Troubleshoot

On demand, when someone reports a problem. Always launch it with a **Limit**: the host with the problem. Findings do not fail this job.

Field	Value
Playbook	<code>playbooks/troubleshoot.yml</code>
Credentials	<code>Linux ssh (sudo)</code>
Survey: <code>ts_area</code>	"What kind of problem?" Choices: <code>overview</code> , <code>disk</code> , <code>performance</code> , <code>service</code> , <code>network</code> , <code>selinux</code> , <code>fapolicyd</code> , <code>login</code> , <code>time</code> , <code>logs</code> , <code>mariadb</code> . Default <code>overview</code>
Survey: <code>ts_service</code>	<i>Text</i> , not required. "Service name (for area = service), e.g. nginx"
Survey: <code>ts_target</code>	<i>Text</i> , not required. "host:port it cannot reach (for area = network)"
Survey: <code>ts_since</code>	<i>Text</i> . "How far back to read logs". Default <code>-2h</code>

Certificate report

Weekly: one table of every certificate on every host, soonest expiry first.

Field	Value
Playbook	<code>playbooks/cert_report.yml</code>
Credentials	<code>Linux ssh (sudo)</code> (+ <code>Keystore password</code> if you use it)
Survey: <code>check_certs_warn_days</code>	<i>Integer</i> . "Warn this many days before expiry". Default <code>30</code>

POA&M status

Weekly. Runs on the controller and reads `poam/poam.csv` from the project.

Field	Value
Playbook	<code>playbooks/poam_status.yml</code>
Credentials	none (+ <code>STIG Manager API</code> for the cross-check)
Privilege escalation	not needed
Limit	empty, and no Prompt on launch (it runs on the controller)
Survey: <code>poam_stigman_api</code>	<i>Text</i> , not required. "STIG Manager API URL (blank = skip the cross-check)"

ServiceNow tickets

Only useful as a workflow step after a check. Runs on the controller.

Field	Value
Playbook	<code>playbooks/servicenow_tickets.yml</code>
Credentials	<code>ServiceNow API</code>

Field	Value
Privilege escalation	not needed
Limit	empty, and no Prompt on launch (it runs on the controller)
Survey	none

ServiceNow health

Hourly. Runs on the controller.

Field	Value
Playbook	<code>playbooks/servicenow_health.yml</code>
Credentials	<code>ServiceNow API</code>
Privilege escalation	not needed
Limit	empty, and no Prompt on launch (it runs on the controller)
Survey	none

Patch hosts

Monthly, inside the *Patch with checks* workflow. Run it as *Check* first: it lists what would be updated and changes nothing.

Field	Value
Playbook	<code>playbooks/patch_hosts.yml</code>
Credentials	<code>Linux ssh (sudo)</code>
Job type	Run, with Prompt on launch ticked, so you can pick <i>Check</i> (a dry run)
Survey: <code>patch_security_only</code>	"Security updates only?" Choices <code>false</code> , <code>true</code> . Default <code>false</code>
Survey: <code>automatic_restarts</code>	"Reboot hosts automatically when the updates need it?" Choices <code>no</code> , <code>yes</code> . Default <code>no</code> (no automatic restarts: the job lists the hosts that need a reboot)

Apply approved ACT fix

Only inside the approval workflow (ADDING_ACT.md). Create it when you add ACT.

Field	Value
Playbook	<code>playbooks/act_fix_approved.yml</code>
Credentials	<code>Linux ssh (sudo)</code> (no ACT key: it runs the approved commands itself)
Job type	Run, with Prompt on launch ticked, so a workflow can run it as <i>Check</i> (a dry run that only shows what it would run)
Variables	leave Prompt on launch unticked

Field	Value
Survey	none

Try each one

With a Limit of one host:

- **Troubleshoot:** `ts_area = disk`. Each command's output appears under a `### what it shows` line.
- **Patch hosts:** Job type **Check**. It lists what would be updated and changes nothing.
- **Certificate report:** read the table under `All certificates on all hosts, soonest expiry first`.

Step 10. ServiceNow

What. An API account, its credential, and a first test.

Why. The tickets step and the ServiceNow health check use ServiceNow's REST Table API with a user name and password. No plugin or collection is installed.

1. Ask the ServiceNow admins for an **integration account** (web service access only) with:
 - `itil` (or a role that can create and update incidents);
 - `mid_server` (read the MID Server list), if you want the MID Server check.
2. Create the **ServiceNow API** credential (step 3) with the instance URL, for example `https://servicenow.example.mil`, and the account.
3. In `inventories/site/group_vars/all.yml`, set who gets the tickets:

YAML

```
servicenow_assignment_group: Linux Operations    # a group that exists in ServiceNow
servicenow_min_severity: warning                # or critical: tickets for critical findings only
```

4. Launch **ServiceNow health** once. It proves the account works: `API answers in N s`, then the MID Servers and their status.
5. Build the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md). Its tickets step then runs after every check.

Try it safely first: launch the workflow **without** the ServiceNow API credential on the tickets template. The step then only prints `NOT CONNECTED - what would be opened` and sends nothing.

Step 11. Workflows and schedules

See WORKFLOWS_AND_SCHEDULES.md: *Daily health*, *Weekly compliance*, *Patch with checks*, *Fix with approval (ACT)*, and when to schedule each.

Step 12. Who can do what

Open a template or workflow › **User Access** (or **Team Access**) › **Add roles**:

Role	Can
Execute	launch it (and answer its survey)
Approve	approve or deny its approval steps (workflows)

Role	Can
Read	see it and its job output
Admin	change it

For example, give the help desk **Execute** on *Troubleshoot*, and only the server team **Execute** on *Patch hosts* and **Approve** on the patch workflow.

If something goes wrong

You see	It means	Do
Project sync fails	wrong URL, branch or Git credential	open the sync job and read the last lines
UNREACHABLE! ... Permission denied (publickey)	SSH key or user wrong for that host	check the Machine credential; test <code>ssh svc_aap@host</code> from a workstation
Missing sudo password / a password is required	sudo wants a password	put it in the Machine credential (Privilege Escalation Password)
Unknown check(s): ...	a survey choice that is not a check name	fix the survey choices (step 6)
the <name> check could not run: ...	that check hit an error. The others still ran	the message says why, often a missing command such as <code>semanage</code>
skipped: <host> is not in mariadb_hosts	the MariaDB check only runs on database hosts	add the host to <code>mariadb_hosts</code> in the inventory
Findings you do not care about	defaults do not fit that host	step 8: change the setting for its group or host
NOT CONNECTED - what would be opened	the tickets template has no ServiceNow API credential	attach it (step 10)
Tickets step fails: No check results reached this job	the check step before it failed before checking any host, or it was launched on its own	read the first red job in the workflow; run the tickets step only inside a workflow
A controller step (tickets, POA&M, ServiceNow health) shows skipping: no hosts matched	its template prompts for a Limit and the workflow passed one	untick Prompt on launch for Limit on that template (step 9)

3. Using the inventory you already have in AAP

You imported your inventory into AAP. You do not need `inventories/site/` in Git, and you can skip the Git inventory parts of `SETUP_AAP.md` (the `cp inventories/example ...` in step 1, and all of step 5). Everything here happens in the AAP web UI. **No YAML file in the repository changes.** What you paste into AAP's **Variables** boxes is YAML, and the blocks to paste are below.

The runbooks need two things from your inventory:

1. **Groups:** which hosts are which (steps 1-2). The safety lists in `playbooks/group_vars/all.yml` already use your group names (`aap`, `sat`, `netapp`...).
2. **Settings (variables):** your LDAP server, your SIEM, the account AAP logs in as, the ServiceNow assignment group... (steps 3-5).

The easy way: your settings files, already named after your groups

The repository has one settings file per AAP group, in `playbooks/group_vars/`. **The file name is the group name:** `mariadb.yml` applies to the hosts in AAP group `mariadb`, and `all.yml` to every host. Ansible reads these files by itself on every run, and the hosts and groups stay in AAP. The files are:

File	For AAP group	What is in it
<code>all.yml</code>	every host (and the ServiceNow / POA&M jobs)	safety lists (<code>aap</code> , <code>sat</code> , <code>idm</code> , <code>netapp</code>); ACT provider and models; placeholders for LDAP/log server, service account, ServiceNow assignment group
<code>stigman.yml</code>	<code>stigman</code>	the containers service watch watches (<code>stigman</code> , <code>nginx</code>); the web certificate
<code>mariadb.yml</code>	<code>mariadb</code>	placeholders: turn the check on, and the container's name
<code>aap.yml</code> , <code>sat.yml</code> , <code>idm.yml</code> , <code>logstash.yml</code>	the same names	disk limits, certificates, services that must run
<code>netapp.yml</code>	<code>netapp</code>	nothing yet (it is protected by <code>all.yml</code>)
<code>rhel8_all.yml</code>	<code>rhel8_all</code>	a placeholder for Python 3.9 (only with a newer AAP)

To change a setting:

1. Open the file in VS Code (for example `playbooks\group_vars\mariadb.yml`).
2. A line that starts with `#` is **off**. To turn a placeholder on, delete the `#` in front of it (and in front of the lines that belong to it), and replace `CHANGE-ME` with your value. Keep the indentation exactly as shown: spaces, never tabs.
3. **Commit** and **Sync Changes**. AAP uses it on the next run: the project syncs itself when **Update revision on launch** is ticked.

Example: MariaDB runs in a container called `snow-mariadb`. In `mariadb.yml`, change

```
check_mariadb_enabled: true
# check_mariadb_container: CHANGE-ME
```

into

YAML

```
check_mariadb_enabled: true
check_mariadb_container: snow-mariadb
```

A group that has no file yet (for example `sn_prod`): create `playbooks/group_vars/sn_prod.yml` with the settings for it. The same name as the AAP group, `.yml` at the end.

One thing the files cannot do: choose which hosts a template runs on. That is decided before any setting is read. Each playbook therefore has a built-in choice, and the job template's own **Variables** box (the *Extra variables* on its Details page, not the inventory's) can change it with one `target:` line:

Job templates	Variables box empty: runs on	To run on something else
Health check, Troubleshoot, Certificate report, Apply approved ACT fix	every host in the template's inventory (the built-in group <code>all</code>)	one run: the Limit at launch (tick Prompt on launch next to Limit). Always: <code>target:</code> <code>"rhel8_all:rhel9_all"</code>
Service watch, Service watch - apply approved fix, STIG Manager - deploy	the group <code>stigman</code>	<code>target: <group></code>
Patch hosts	the group <code>patch_hosts</code> only (no such group: it patches nothing)	always set it , for example <code>target:</code> <code>"rhel8_all:rhel9_all"</code> , and pick the hosts with the Limit at launch

`target` and the Limit take group names, host names, or several joined with `:. stigman:mariadb` means either group, `rhel9_all:&sn_prod` only hosts in both, and `rhel9_all:!sn_dev` all of `rhel9_all` except `sn_dev`. Put a space after `target:`, quote a value that has a `:` in it, and use the group name exactly as the inventory's **Groups** tab shows it (`rhel9_all`, not `rhel9`).

Is every host in this inventory a Linux server? If some are not (firewalls, switches, appliances), an empty Variables box tries them too: they show as failed, and in a workflow that opens ServiceNow tickets each of them gets a ticket (a host that never reports is ticketed), on every run. Then put `target:`
`"rhel8_all:rhel9_all"` on those templates **before** you schedule anything.

These files are yours. The update script adds a settings file you do not have yet, and never changes one you already have. A new release can therefore never overwrite your settings.

Or put the same lines in AAP instead. Everything below explains the AAP Variables boxes, which take exactly the same lines. **Keep each setting in one place.** If the same setting is in both an AAP group and a file here, **the file here wins**.

How variables work in AAP (two minutes)

A variable is a setting with a name, such as `check_disk_warn_pct: 85`. Where you type it decides which hosts get it:

Where in AAP	Applies to	The Git-file equivalent
Inventories › your inventory › Edit inventory › Variables	every host in the inventory	<code>group_vars/all.yml</code>

Where in AAP	Applies to	The Git-file equivalent
... › Groups › a group › Edit group › Variables	the hosts in that group (and in its child groups)	<code>group_vars/<group>.yaml</code>
... › Hosts › a host › Edit host › Variables	that one host	<code>host_vars/<host>.yaml</code>

- **The more specific one wins:** host beats group, a child group beats its parent group, and a group beats the inventory. A survey answer, or the template's own **Variables**, beats all of them.
- **Anything you do not set uses the default** in `roles/<role>/defaults/main.yaml`. Every setting is explained there. Do not edit those files: set the value in AAP instead. A new release would overwrite edits in the repository, but never your AAP variables.
- The boxes take YAML. Keep the indentation exactly as shown (spaces, not tabs). AAP refuses to save YAML it cannot read.

Step 1. Look at what you have

Inventories › your inventory: open the **Groups** tab, then the **Hosts** tab. Write down which of your groups (or hosts) are:

- all your Linux servers;
- the AAP server(s);
- the NetApp Console host;
- the STIG Manager host(s);
- the ServiceNow database host(s);
- the ServiceNow MID Server host(s);
- the servers monthly patching may touch.

On the old ansible-core machine, `ansible-inventory -i <your inventory> --graph` prints the same tree.

Step 2. Create the groups the runbooks use

A host can be in many groups. Creating these groups leaves your existing groups as they are.

Create group	Put in it	Why	Needed?
<code>aap_hosts</code>	the AAP server(s)	safety: ACT never fixes anything here by itself, and patching never reboots it	yes
<code>netapp_console_hosts</code>	the NetApp Console host	safety: ACT only diagnoses here (vendor-managed)	yes , if you have one
<code>no_patch</code>	the groups <code>aap_hosts</code> and <code>netapp_console_hosts</code> (as children)	safety: <i>Patch hosts</i> skips these even when a job targets them	yes
<code>patch_hosts</code>	the servers monthly patching may touch	<i>Patch hosts</i> only patches these	only for patching

Create group	Put in it	Why	Needed?
<code>mariadb_hosts</code>	the ServiceNow database host(s)	the MariaDB check runs only here	only for the MariaDB check
<code>stigman</code>	the STIG Manager host(s)	service watch and STIG Manager deploy run here	only for those

The clicks:

1. **Inventories › your inventory › Groups › Create group.** Type the **Name** (exactly as in the table), then **Create group**.
2. Add hosts: open the new group › **Hosts** tab › **Add existing host** › tick the hosts › **Add**.
3. Or add one of your groups inside it: open the new group › **Related Groups** tab › **Add existing group** › tick your group › **Add**. Its hosts are now in the new group too.

Keep your own names instead? If you already have, say, an `aap` group and do not want a second group, tell the runbooks your names. Put these in the inventory **Variables** (step 3):

YAML

```
site_act_diagnose_only_groups: [aap, netapp]    # ACT never fixes these
patch_never_reboot_groups: [aap]              # patching never reboots these
patch_never_patch_groups: [aap, netapp]       # patching skips these
```

The checks run on every host of the inventory. To keep them to your Linux servers, put `target: linux_servers` (your all-Linux group) in the **Variables** of the Health check, Troubleshoot, Certificate report and Apply approved ACT fix templates.

Step 3. Settings for every host (inventory variables)

Inventories › your inventory › Edit inventory › Variables. If something is already there, add below it; do not delete what is there. Paste, then change every line marked **CHANGE**:

```
--- what every host must be able to reach, from itself ---

check_network_dns_names: [ldap.yoursite.mil]    # CHANGE: names that must resolve
check_network_tcp:                               # CHANGE: services every host needs
  - {name: LDAP, host: ldap.yoursite.mil, port: 636}
check_logging_remote:                             # CHANGE: your SIEM / log collector
  - {name: SIEM, host: siem.yoursite.mil, port: 6514}

# ---- the account AAP logs in as ----
check_accounts_nopasswd_allowed: [svc_aap]      # CHANGE: your AAP service account
check_accounts_expiry_watch: [svc_aap]         # CHANGE: the same account

# ---- STIG ----
check_fapolicyd_required: true                  # false if some hosts do not run
  fapolicyd

# ---- where tickets go ----
servicenow_assignment_group: Linux Operations  # CHANGE: a group that exists in
  ServiceNow
servicenow_min_severity: warning               # or critical: tickets for critical
  only
```

The ACT provider and models are **not** set here: they are already in `playbooks/group_vars/all.yml`, and that file beats this box, so a value here would be ignored. Change them in the file (ADDING_ACT.md, step 2).

Not sure about a value yet? Leave the line out. The check then uses its default, or checks nothing for it (no `check_network_tcp` means no TCP checks).

Step 4. Settings for particular groups

Open the group › **Edit group** › **Variables**, paste, change the **CHANGE** lines, **Save**. Put these on your own groups or on the groups from step 2. Both work.

The AAP server(s), for example on `aap_hosts`:

```
YAML
check_disk_overrides:
  /var: {warn: 80, crit: 90}          # AAP keeps job output and container images under /var
```

The STIG Manager host(s), on `stigman`:

```
YAML
check_certs_endpoints:                # the certificate browsers actually see
- {name: STIG Manager, host: 127.0.0.1, port: 443, servername: stigman.yoursite.mil} # CHANGE
check_certs_files: [/etc/stigman/*.crt, /etc/stigman/*.pem] # CHANGE:
  where your cert files are
watch_containers:                    # service watch, dependencies first: container names
- name: stigman                      # (sudo podman ps -a) or service names without .service;
- name: nginx                        # how each is run (systemd unit or podman) is found by
  itself
```

If STIG Manager runs as systemd units (`systemctl list-units | grep -i stig` shows them), also add them, so the services check watches them: `check_services_required_extra: [<unit>, <unit>]`.

The ServiceNow database host(s): also add this group to `mariadb_hosts` (step 2).

```
YAML
check_mariadb_container: servicenow-mariadb # CHANGE: the container name from `sudo podman ps`
```

Then attach the **MariaDB monitor** credential to the Health check template. Container images give root a password, so the check logs in with a monitoring account instead. RUNBOOKS.md has the one-line GRANT to ask the DBA for.

The ServiceNow MID Server host(s):

```
YAML
check_services_required_extra: [mid] # CHANGE: its unit name (systemctl list-units |
  grep -i mid)
check_certs_keystores:
- path: /opt/servicenow/mid/agent/security/agent_keystore # CHANGE: your MID Server's path
```

The NetApp Console host: nothing is needed. It is checked like every host; being in `netapp_console_hosts` and `no_patch` keeps automation from changing it.

Step 5. Settings for one host (only when one host is different)

Hosts › the host › Edit host › Variables. For example, a host whose `/data` is always nearly full on purpose:

YAML

```
check_disk_overrides:
  /data: {warn: 95, crit: 98}
```

Or a host with a known-broken vendor unit you want ignored:

YAML

```
check_services_ignore_failed: ['^vendor-agent']
```

Step 6. Use your inventory in the job templates

On every job template (and workflow), set **Inventory** to your inventory. That is the only link between them. Everything else (project, playbook, credentials, survey) is as in SETUP_AAP.md, steps 6 and 9.

Step 7. Check that it worked

1. Launch **Health check** with **Limit** = one server and the survey set to **daily**.
2. In **Report | result for this host**, look for your own values. A network finding names *your* LDAP server; an accounts finding names *your* service account. If a value is not there, the variable did not reach the host: check its spelling, and which group or host it is on.
3. **skipping: no hosts matched** (a green job that did nothing) means the Limit or **target** names a group or host this inventory does not have. Check the spelling against the inventory's **Groups** tab, and that the template uses the right **Inventory**.
4. **skipped: <host> is not in mariadb_hosts** means the MariaDB check needs that host in **mariadb_hosts**.
5. Findings you do not care about: change the setting for that group or host (steps 4-5), not the code.

Which YAML files in the repository do I change?

File	Change it?
<code>inventories/example/...</code>	No. Only a reference to copy blocks from; AAP does not read it
<code>roles/*/defaults/main.yml</code>	No. Read it to see every setting; set values in AAP instead
<code>playbooks/*.yml</code>	No
<code>aap/credential_types/*.yml</code>	No. Paste their contents into AAP (credential types)
<code>ansible.cfg</code>	No. AAP ignores its <code>inventory =</code> line and uses the template's inventory
<code>poam/poam.csv</code>	Yes. Your POA&M export (see <code>poam/README.md</code>)

4. Workflows and schedules

A **workflow** chains job templates. Each arrow says when the next step runs:

- **Run on success:** only if the step before it succeeded.
- **Run on fail:** only if it failed.
- **Run always:** either way.

The checks *fail on purpose* when they find something. So "Run on fail" means "a problem was found", and "Run always" means "whatever happened". A step also receives what the steps before it published (**artifacts**). That is how the tickets step learns what the health check found.

This chapter first shows how to build any workflow, then a catalog of ready-made workflows for Linux, for Windows, and for both, and how to schedule them.

How to build any workflow, click by click

1. **Automation Execution › Templates › Create template › Create workflow job template.**
2. **Name** (e.g. **Daily health**), **Organization**.
 - **Inventory: leave it empty.** Each step then uses its own template's inventory, so one workflow can hold Linux and Windows steps. A workflow inventory only reaches steps whose template prompts for the inventory.
 - Tick **Prompt on launch** next to **Limit** if you want to pick hosts at launch.
 - Leave **Variables** empty: workflow variables override every step's own answers.
3. **Create workflow job template.** The **workflow visualizer** opens.
4. **The first step: Add step › Node type:**
 - *Job Template:* one of the templates (the usual case);
 - *Approval:* a person must click Approve (give it a **Timeout**);
 - *Workflow Job Template:* another workflow, as one step;
 - *Project Sync / Inventory Source Sync:* refresh the code or the inventory first.

Pick the template, then fill in **this step's** answers:

- **Survey answers**, e.g. `health_checks = daily`;
- **Job type** **Check** for a dry run (DRY_RUNS.md);
- a **Limit**.

The wizard only offers what the template asks in its survey or has **Prompt on launch** ticked for. › **Finish**.

5. **Next steps:** hover over a box › **Add step and link** (the plus sign) › the node, and the **link type** (success / fail / always).
 - **Two things at once:** add a second first step with **Add step**. Both start together.
 - **Two arrows into one step:** open the step › **Convergence:** *Any* runs it when one parent finishes as linked, *All* waits for all of them.
6. **Save** (top right). Closing without saving throws the graph away.
7. **Try it:** launch it once with the risky steps set to **Check** (step 4), then set them back.

Who may approve: open the workflow › **User Access** (or **Team Access**) › **Add roles** › **Approve**. Pending approvals are in **Automation Execution › Administration › Workflow Approvals**, and in the workflow

job's view.

Which templates fit where:

Template	Fails when	Publishes (for the next step)	Typical next step
Health check / Windows health check	a finding (<code>site_fail_on</code>), or ACT proposes a fix	the findings (<code>site_health</code>)	ServiceNow tickets (always); an approval (on fail)
Certificate report / Windows certificate report	a certificate expires soon	the findings	ServiceNow tickets (always)
POA&M status	an item is overdue / due soon	the findings	ServiceNow tickets (always)
Troubleshoot / Windows troubleshoot	never for findings (a look, not a verdict)	the findings	a health check, tickets if you want them
Patch hosts / Windows patch	a host fails, or its services do not come back	-	a health check (always)
Apply approved ACT fix	the problem is still there	the findings of the re-check	ServiceNow tickets (always)
Service watch - check	a container is down (NEEDS APPROVAL)	the fix to approve	an approval (on fail)
ServiceNow tickets	problems exist (it opened or updated tickets)	-	- (the end)

Linux workflows

1. Daily health



Daily health: every run hands its findings to the tickets step, healthy or not.

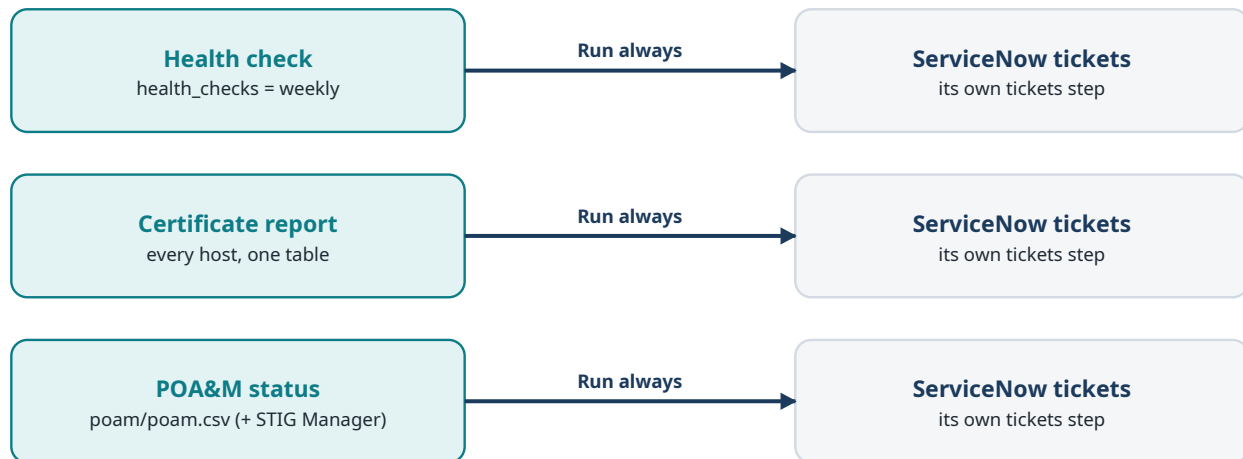
Step	Template	Answers	Link
1	Health check	<code>health_checks = daily</code>	(start)
2	ServiceNow tickets	none	Run always , from step 1

Why "always". The tickets step opens tickets for what failed, and also notes the tickets of problems that cleared. That needs the healthy runs too. It fails when problems exist, so the workflow shows red until they are fixed.

Schedule: every day at 06:00.

2. Weekly compliance

Three independent chains in one workflow. Add each first box with **Add step** (not "Add step and link"), so they all start together:

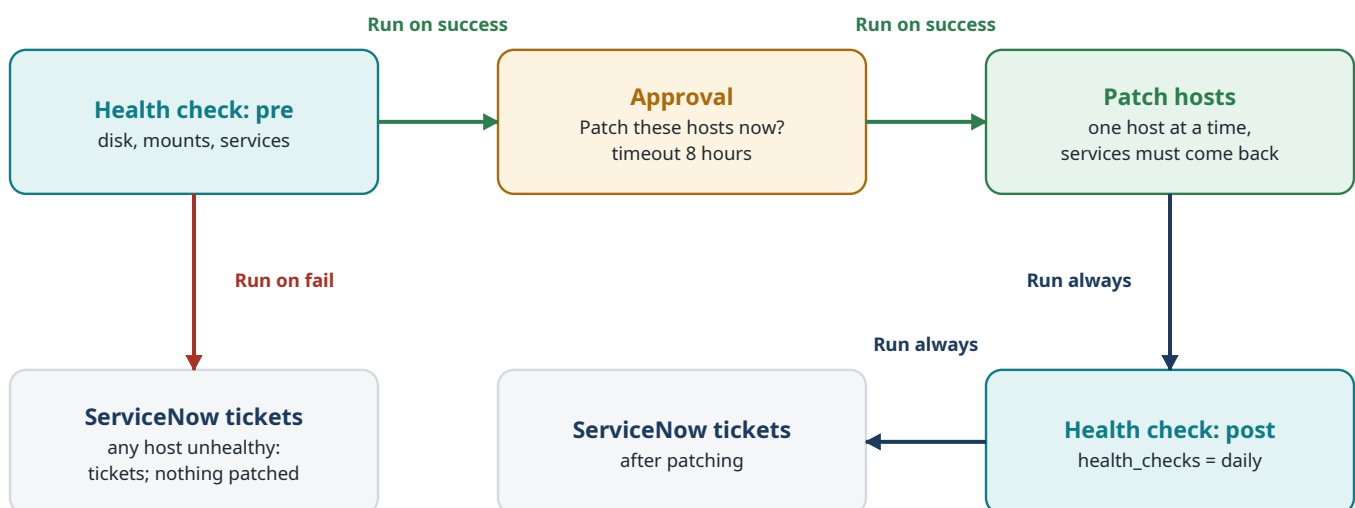


Weekly compliance: three independent chains, each with its own tickets step (a step with two parents would get only one parent's findings).

Give each chain **its own** tickets box. A box with two parents gets only one parent's findings: the artifacts overwrite each other.

Schedule: Mondays at 07:00.

3. Patch with checks



Patch with checks: every host healthy first (one failing host stops the whole run), a person approves, patch one host at a time, then check again.

Step	Template	Answers	Link
1	Health check	<code>health_checks = disk, mounts, services</code>	(start)

Step	Template	Answers	Link
2	Approval	Name Patch these hosts now? , Timeout 8 hours	Run on success , from 1
3	Patch hosts	patch_security_only as you like; automatic_restarts (see below)	Run on success , from 2
4	Health check	health_checks = daily	Run always , from 3
5	ServiceNow tickets		Run always , from 4
6	ServiceNow tickets		Run on fail , from 1 (tickets for what the pre-check found)

Why these links:

- **Healthy first.** Patching a host that is already failing makes the next problem impossible to diagnose. The pre-check must pass before anyone is asked. **If any one host fails it, the whole run stops before the approval and no host is patched.** Fix that host (the ticket says what is wrong), or launch again with a Limit that leaves it out. Warnings count too while **site_fail_on** includes **warning**. A disk at 86% blocks patching, so clear it before the maintenance window.
- **A person approves.** The approval step waits (up to its **Timeout**). Deny, or no answer, stops the workflow and nothing is changed.
- **Always check after.** The post-check runs whatever happened in patching, so a half-patched host is still reported.

No automatic restarts unless you ask for them. Patching installs the updates; a host that then needs a reboot is **not** rebooted, and the job says so. The post-check's **patching** finding ("reboot needed") keeps it visible until someone reboots it. To let the job reboot hosts that need it, give step 3 **automatic_restarts** = **yes**. Add a survey question for it on the Patch hosts template (*Multiple Choice, no / yes*, default **no**), or put **automatic_restarts: true** in the step's variables. Hosts in **patch_never_reboot_groups** are never rebooted either way.

Launch with **Limit** **patch_hosts**, or one group at a time (**stigman** this week, **servicenow_mid_hosts** next week). **Schedule:** monthly, in your maintenance window, for example the second Wednesday at 20:00. The approval then waits for the on-call person.

4. Patch with a dry run first

The same as 3, but the approver sees exactly what will be installed before approving:

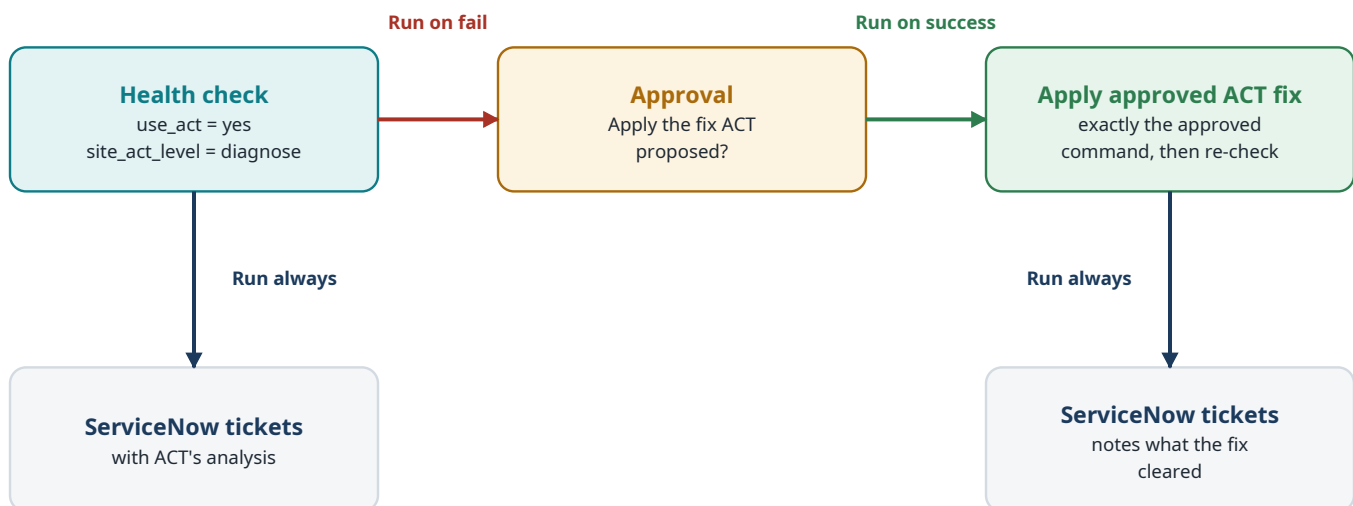
```
[Health check: pre]      health_checks = disk, mounts, services
| Run on success
[Patch hosts]           Job type = Check: lists what it would install
| Run on success
[Approval]              "Install what the dry run listed?"
| Run on success
[Patch hosts]           Job type = Run
| Run always
[Health check: post]    health_checks = daily
| Run always
[ServiceNow tickets]
```

Step	Template	Answers	Link
1	Health check	<code>health_checks = disk, mounts, services</code>	(start)
2	Patch hosts	Job type <code>Check</code>	Run on success , from 1
3	<i>Approval</i>	Name <code>Install</code> what the dry run listed?. Description: Open the "Patch hosts" dry-run job: it lists every package per host.	Run on success , from 2
4	Patch hosts	Job type <code>Run</code> ; <code>automatic_restarts</code> as you decide	Run on success , from 3
5	Health check	<code>health_checks = daily</code>	Run always , from 4
6	ServiceNow tickets		Run always , from 5

Needs **Prompt on launch** for **Job type** on the Patch hosts template (steps 2 and 4 set it).

5. Fix with approval (ACT)

For when you use ACT. See `ADDING_ACT.md` for the three levels.



Fix with approval (ACT): the check job fails with `NEEDS APPROVAL` only when ACT proposed a fix; deny or timeout changes nothing.

Step	Template	Answers	Link
1	Health check (with the ACT survey questions)	<code>use_act = yes, site_act_level = diagnose</code>	(start)
2	ServiceNow tickets		Run always , from 1
3	<i>Approval</i>	Name <code>Apply the fix ACT proposed?</code> , Timeout 30 minutes. Description: Read ACT's report and the <code>NEEDS APPROVAL</code> line in the health check job first.	Run on fail , from 1

Step	Template	Answers	Link
4	Apply approved ACT fix		Run on success , from 3
5	ServiceNow tickets		Run always , from 4 (notes the tickets whose problem the fix cleared)

The approver reads, in the health check job, **ACT | what ACT says** (root cause, evidence, the fix) and the red **NEEDS APPROVAL ... ACT proposes: <command>** line. **That exact command** is all the apply step can run: it runs it itself (no model, no key), stops at the first command that fails, and runs the same checks again. The tickets carry ACT's analysis too.

Rehearse it: see ACT's fixes, change nothing.

- **One job, no workflow.** Launch **Health check** (or **Troubleshoot**) with `use_act = yes` and `site_act_level = diagnose`. ACT investigates with read-only commands and proposes fixes; every change is refused. The job ends red with **NEEDS APPROVAL ... ACT proposes: <commands>**: that list is what would run. Nothing on the host changed.
- **The whole workflow, as a dry run.** On the **Apply approved ACT fix** template (and *Service watch - apply approved fix*) tick **Prompt on launch** next to **Job type**. In the workflow visualizer click the apply box › **Edit** (pencil) › **Job type Check** › **Save**. Now run the workflow and approve: the apply job prints **DRY RUN on <host>: approved, and would run: <commands>. Nothing was changed (Check mode).** and ends green. When you trust it, set that node's **Job type** back to **Run**.

6. Security posture (weekly)

The STIG-related checks, and the POA&M list, with tickets:

```
[Health check] --Run always--> [ServiceNow tickets]
selinux, fapolicyd, auditd, accounts
[POA&M status] --Run always--> [ServiceNow tickets]
```

Step	Template	Answers	Link
1	Health check	<code>health_checks = selinux, fapolicyd, auditd, accounts</code>	(start)
2	ServiceNow tickets		Run always , from 1
3	POA&M status		(start, with Add step)
4	ServiceNow tickets		Run always , from 3

Schedule: Wednesdays 07:00. Use it instead of the `weekly` set in *Weekly compliance* if your security team wants these on their own day.

7. Certificate watch (daily)

Certificates expire on a day, not in a week:

```
[Certificate report] --Run always--> [ServiceNow tickets]
```

Schedule: every day 06:30. Set `check_certs_warn_days` (survey) to 30 so tickets start a month ahead. The de-duplication keeps it to one ticket per certificate.

8. MariaDB watch (hourly)

```
[Health check] --Run always--> [ServiceNow tickets]
disk, services, mariadb
```

Step	Template	Answers	Link
1	Health check	<code>health_checks = disk, services, mariadb; Limit mariadb</code>	(start)
2	ServiceNow tickets		Run always, from 1

Schedule: hourly. It is quick, and a ticket comes within the hour of a problem.

9. Investigate a host (on demand)

One launch gives the troubleshooting commands, ACT's explanation and every check:

```
[Troubleshoot] --Run always--> [Health check]
ts_area, use_act = yes,          health_checks = all
site_act_level = explain
```

Step	Template	Answers	Link
1	Troubleshoot	<code>ts_area = overview</code> (or asked at launch), <code>use_act = yes, site_act_level = explain</code>	(start)
2	Health check	<code>health_checks = all</code>	Run always, from 1

Give the workflow **Prompt on launch** for **Limit** and launch it with one host. No tickets: you are already looking. Add `[ServiceNow tickets]` after step 2 (**Run always**) if you want them.

10. STIG Manager deploy with checks

```
[STIG Manager - deploy]      Job type = Check (+ Show changes): what would change
| Run on success
[Approval]                  "Apply the changes the dry run listed?"
| Run on success
[STIG Manager - deploy]      Job type = Run
| Run always
[Service watch - check]
| Run always
[Health check]              health_checks = certs, services
```

Step	Template	Answers	Link
1	STIG Manager - deploy	Job type Check (and Show changes)	(start)
2	<i>Approval</i>	<code>Apply the changes the dry run listed?</code>	Run on success, from 1
3	STIG Manager - deploy	Job type Run	Run on success, from 2
4	Service watch - check		Run always, from 3
5	Health check	<code>health_checks = certs, services</code>	Run always, from 4

The dry run shows which files and units would change, and how. The approver approves only changes they expected. (This dry run has not been tried on a real server yet: run step 1 on its own once, on the test server, before you build the workflow.)

11. Service watch

The approval or self-heal workflow for containers: SERVICE_WATCH_DEMO.md, Part 6.

Windows workflows

Build them the same way, with the Windows templates (WINDOWS.md). The ServiceNow tickets step is the same template as for Linux: it runs on AAP itself.

12. Windows daily health

```
[Windows health check] --Run always--> [ServiceNow tickets]
health_checks = daily
```

Schedule: every day 06:15.

13. Windows weekly compliance

```
[Windows health check] --Run always--> [ServiceNow tickets]
health_checks = weekly
[Windows certificate report] --Run always--> [ServiceNow tickets]
```

Schedule: Mondays 07:15.

14. Windows patch with a dry run first

```
[Windows health check: pre] health_checks = disk, services
| Run on success
[Windows patch] Job type = Check: lists the updates Software Center has
| Run on success
[Approval] "Install what the dry run listed?"
| Run on success
[Windows patch] Job type = Run (automatic_restarts?)
| Run always
[Windows health check: post] health_checks = daily
| Run always
[ServiceNow tickets]
```

Step	Template	Answers	Link
1	Windows health check	<code>health_checks = disk, services</code>	(start)
2	Windows patch	Job type <code>Check</code>	Run on success , from 1
3	<i>Approval</i>	<code>Install what Software Center offers (listed in the dry run)?</code> , Timeout 8 hours	Run on success , from 2
4	Windows patch	Job type <code>Run</code> ; <code>automatic_restarts</code> as you decide	Run on success , from 3
5	Windows health check	<code>health_checks = daily</code>	Run always , from 4
6	ServiceNow tickets		Run always , from 5

No automatic restarts unless step 4 gets `automatic_restarts = yes`: the servers that need a restart are listed, and the post-check's `patching` finding ("a restart is pending") keeps them visible. **Schedule:** inside the servers' ConfigMgr maintenance window. Outside it, ConfigMgr holds the updates and the patch step stops and says so.

15. ACT for Windows rollout

```
[ACT for Windows - install]      Job type = Check: would install / update / already current
|   Run on success
[Approval]
|   Run on success
[ACT for Windows - install]      Job type = Run
```

The dry run lists, per server, whether ACT would be installed, updated, or is already current. Launch it with a Limit (a group) to roll out group by group. Run it again after every update of this repository.

16. Windows: investigate a server (on demand)

```
[Windows troubleshoot] --Run always--> [Windows health check]
ts_area                health_checks = all
```

Launch with one server in the Limit.

Linux and Windows together

17. Everything, every morning

One workflow, two independent chains (each first box added with **Add step**). Each step uses its own template's inventory, so leave the workflow's Inventory empty:

```
[Health check]      --Run always--> [ServiceNow tickets]
health_checks = daily
[Windows health check] --Run always--> [ServiceNow tickets]
health_checks = daily
[Certificate report] --Run always--> [ServiceNow tickets]
```

Schedule: every day 06:00. It replaces workflows 1, 7 and 12.

Schedules

What. A schedule launches a template or workflow at set times.

How. Open the workflow (or template) › **Schedules** tab › **Create schedule**:

1. **Name, Start date/time, Time zone** (use your site's, not UTC, so "06:00" means 06:00).
2. **Repeat frequency** and its details: *Day* (every 1 day), *Week* (on Monday), *Month* (on the second Wednesday)...
3. **Prompts:** the survey answers, the Limit and the Job type **for this schedule**. Without them, the schedule uses the template's **defaults**. A schedule can only change what the template **prompts on launch** or asks in its survey.
4. **Finish.** The schedule's page shows the next run times. Check them once.

What	When	Launches
Daily health (or <i>Everything, every morning</i>)	every day 06:00	workflow 1 (or 17)
Certificate watch	every day 06:30	workflow 7
MariaDB watch	hourly	workflow 8
ServiceNow health	hourly	template <i>ServiceNow health</i> , with an email notification on failure
Weekly compliance	Monday 07:00	workflow 2
Security posture	Wednesday 07:00	workflow 6
Patch preview	the Monday before patch night	template <i>Patch hosts</i> , Prompts › Job type Check
Patch with checks	monthly, in your window	workflow 3 or 4
Service watch	every 15 minutes	workflow <i>Service watch - approve or self-heal</i>
Windows daily health / weekly compliance	06:15 daily / Monday 07:15	workflows 12 and 13
Windows patch	monthly, in the ConfigMgr maintenance window	workflow 14

Notifications (optional). **Automation Execution › Administration › Notifiers › Create notifier** (type *Email*, your mail relay). Then on a template or workflow › **Notifications** tab › switch on **Failure**. Use it where a ticket cannot help. ServiceNow health is the main case: if ServiceNow is down, it cannot take a ticket about itself.

Things that trip people up

- **Red is not broken.** A check job with findings is red by design. Read **Report | result for this host:** findings mean the playbook worked.
- **Save the visualizer.** Closing without **Save** loses the graph.
- **One tickets box per check box.** Artifacts from two parents overwrite each other.
- **A step can only be set to Check** (or given a Limit, or survey answers) if its template has **Prompt on launch** for it, or a survey.
- **Schedules use survey defaults** unless the schedule sets its own answers (Prompts step).
- **Approvals need a timeout.** Otherwise a forgotten approval keeps the workflow open forever. After the timeout it counts as denied.
- **Workflow variables override node answers.** Leave the workflow's **Variables** empty, and set answers on each step.
- **Leave the workflow's Inventory empty** unless every step should use it: it only reaches steps whose template prompts for the inventory, and a Linux inventory on a Windows step matches no host.
- **Limit on the workflow** applies to every step whose template prompts for a Limit. The controller-side templates (ServiceNow tickets, ServiceNow health, POA&M status) must **not** prompt for one, or a workflow Limit makes them match no host and silently do nothing.

- **A tickets step fails when nothing reached it.** If the check step died before checking any host (credential, project sync, execution environment), the tickets step fails too, so a broken schedule shows red instead of green. Read the first red job.
- **No automatic restarts** unless a patch step gets `automatic_restarts = yes`.

5. Dry runs: try anything, change nothing

A **dry run** runs a job without changing anything. Ansible calls it *check mode*; AAP calls it **Job type: Check**. Every step that would change something only says what it *would* change. The steps that only read (the health checks, looking at logs, asking ServiceNow what is open) still run, so the answers are real.

Use a dry run the first time you use any template, on one host, and before every change you are not sure about: patching, applying an approved fix, installing ACT.

Where to set it

There are five places. Pick the one that fits:

Where	How	Good for
1. The template	Template › Edit › Job type Check	a template that is <i>always</i> a dry run, e.g. a copy named Patch hosts (dry run) that more people may launch
2. At launch	Template › Edit › tick Prompt on launch next to Job type . Then at launch, pick Check or Run	the usual way: the same template, you choose each time
3. A workflow step	(needs 2 on the template) Workflow › Visualizer › click the step › Edit (pencil) › Job type Check › Save	rehearsing a whole workflow, or a "dry run first, then real" workflow
4. A schedule	(needs 2) Template or workflow › Schedules › the schedule › Prompts › Job type Check	a regular preview, e.g. "what would patch night install" a week before
5. Command line	ansible-playbook ... --check (add --diff to see file changes)	testing from a shell

Show changes. Tick **Show changes** on the template (or **--diff**) and a dry run also shows the line-by-line changes it would make to files, for example in STIG Manager's configuration.

How to tell. The job's **Details** tab says **Job type: Check**. Our playbooks also print **DRY RUN (Check mode): would ...** where it matters.

What each template does in a dry run

Template	In a dry run it...
Health check, Troubleshoot, Certificate report	runs exactly as usual : they only read, so you get the same findings. Nothing is written: no syslog lines, no disk-forecast sample, and on Windows no event log entries
POA&M status	as usual: reads the CSV and asks STIG Manager
ServiceNow tickets	asks ServiceNow what is already open, then prints would open: ..., would add a note to: ..., would resolve: Nothing is sent
ServiceNow health	as usual: only reads
Patch hosts (Linux)	lists the packages dnf would update. Installs nothing, restarts nothing
Apply approved ACT fix	DRY RUN on <host>: approved, and would run: <commands>. Runs nothing
Service watch - check	checks the containers and shows what it would propose. Writes no incident, does not call ACT
Service watch - apply approved fix	DRY RUN on <host>: approved, and would run: <commands>
STIG Manager - deploy	says which files and units would change (with Show changes: how), and verifies nothing (nothing was started). Changes nothing. <i>Not yet tried on a real server: try it once on the test server</i>
Windows health check, troubleshoot, certificate report, connection test	as usual: they only read
Windows patch	asks Software Center to rescan (without starting anything) and prints DRY RUN (Check mode): would install <updates>
ACT for Windows - install	DRY RUN: would install ACT in C:\ProgramData\act because ..., or that it is already current

ACT does not run in a dry run. It would call the model, and ACT could change things. A dry run with `use_act = yes` reports the checks and says ACT gave no result. To see ACT's ideas without changes, do a *real* run with `site_act_level = diagnose`: ACT then only reads and proposes (ADDING_ACT.md).

Examples

Before patching one server. Launch **Patch hosts** › **Job type** **Check**, **Limit** **web01**. The output lists every package that would be updated. Nothing is installed.

```
TASK [patch : Patch | what was updated (dry run - what WOULD be updated)]
ok: [web01] => {"msg": "DRY RUN (Check mode), nothing was installed - would make 4 change(s):
  Installed: expat-2.5.0-6.el9_8.5.x86_64; Installed: libxml2-2.9.13-14.el9_8.5.x86_64;
  Removed: libxml2-2.9.13-14.el9_8.4.x86_64; Removed: expat-2.5.0-6.el9_8.3.x86_64"}
```

(**Removed** is the old version that the new one replaces.)

Before patch night, every month. On **Patch hosts** add a schedule for the Monday before patch night, with **Prompts** › **Job type** **Check**. The job output shows what patch night will install on every host, in time to exclude something.

A dry-run copy for the team. Template **Patch hosts** › ... › **Copy**. Rename the copy **Patch hosts (dry run)**, set **Job type** **Check**, untick **Prompt on launch** for it, and give your team **Execute** on the copy only. They can preview; only you can patch.

Which tickets would a run open? In the *Daily health* workflow click the **ServiceNow tickets** step › **Edit** › **Job type** **Check**. Run the workflow: the tickets job prints what it would open, note and resolve. Set it back to **Run** when the tickets look right.

A Windows server, before its first real patch. Launch **Windows patch** › **Job type** **Check**, **Limit** one server:

```
"DRY RUN (Check mode): would install 2026-09 Cumulative Update (KB5040001); Malicious Software Removal Tool (KB890830)"
```

The approve-and-fix path, end to end. See WORKFLOWS_AND_SCHEDULES.md, "Fix with approval (ACT)": set the apply step to **Check**, run the workflow, approve. The apply job shows what it would run.

From a shell (on a machine with Ansible and this repository):

```
shell
ansible-playbook -i <inventory> playbooks/patch_hosts.yml --check --diff -l web01
ansible-playbook -i <windows inventory> playbooks/win_patch.yml --check -l web01
```

What a dry run cannot tell you

- Whether the updates will **install cleanly**, or how long they take: it only lists them.
- Whether a **restart** would succeed: nothing restarts in a dry run.
- Anything that **depends on an earlier change**. In a dry-run workflow the post-check after "patching" still sees the unpatched host, because nothing was patched.
- What **ACT** would say (see above).

6. The runbooks: every check and what to do

For each runbook: what it answers, its settings (defaults in `roles/<role>/defaults/main.yml`, change them in the inventory), and each finding with what usually fixes it. The `look:` command printed with a finding is always the first thing to run.

Health check (playbooks/health_check.yml)

Runs the checks you pick on each host: a list of names, or the shortcuts:

- `daily` = disk, mounts, services, performance, time, network, logging, mariadb
- `weekly` = selinux, fapolicyd, auditd, accounts, certs, patching
- `all` = both

A host with a finding at a severity in `site_fail_on` (default: critical and warning) fails. That is what a workflow reacts to. A check that itself breaks becomes a warning finding (the `X check could not run: ...`) and the other checks still run.

disk

Finding	Means	Usually
<code>/var is 96% full</code>	local filesystem space (<code>check_disk_warn_pct 85</code> , <code>check_disk_crit_pct 95</code>)	find what grew with the <code>look:</code> command: old logs, core dumps, a runaway log file. <code>journalctl --vacuum-size=500M</code> for the journal
<code>... has used 97% of its inodes</code>	too many small files (space may still be free)	usually a directory with millions of small files (session/cache/spool); <code>look:</code> finds it
<code>/var will be full in about 3.2 days</code>	growth-rate forecast from earlier runs (needs a day of history)	same as above, before it becomes an outage
<code>could not read filesystem usage</code>	<code>df</code> failed or timed out	a hung mount; see the mounts check

Per-mount limits: `check_disk_overrides: {/var/log/audit: {warn: 70, crit: 85}}`. The forecast keeps a small history in `/var/lib/site-health/disk-history.json` (the only file the checks write; a Check-mode run does not write it).

mounts

Finding	Means	Usually
<code>/data (xfs) is in /etc/fstab but is not mounted</code>	it did not mount at boot, or was unmounted	<code>journalctl -b</code> for the mount error; a missing disk or LUN; a typo in fstab
<code>/var is mounted READ-ONLY</code>	the kernel remounted it read-only after disk/filesystem errors	urgent: <code>journalctl -k</code> for I/O errors; storage team; plan an fsck
<code>network mount /mnt/share does not answer within 5s</code>	NFS/CIFS server down, network, or a stale handle	check the server and the network; diagnose only (NetApp is vendor-managed)
<code>network mount ... is 96% full</code>	the share is full	the storage owner

services

Finding	Means	Usually
<code>systemd unit X has FAILED</code>	a unit crashed or exited with an error	<code>systemctl status X</code> , <code>journalctl -u X</code> ; fix the cause, then <code>systemctl restart X</code>
<code>required service X is inactive</code>	a service that must run (<code>check_services_required + _extra</code>) is stopped	same; find out why it stopped before starting it
<code>required service X is not installed</code>	the unit does not exist here	remove it from the list for this group, or install it
<code>required service X is disabled</code>	it will not start after a reboot	<code>systemctl enable X</code>
<code>X has restarted 5 times</code>	a restart loop: it keeps crashing	its log shows the crash

Per group: `check_services_required_extra: [mariadb]`. To ignore a failed unit: `check_services_ignore_failed: ['^dnf-makecache']`.

performance

Finding	Means	Usually
15-minute load is 14.2 on 4 CPUs (3.55 per CPU)	more work than CPUs, for 15 minutes	the top CPU processes are in the finding; a runaway job, or undersized
only 4% of memory is available	memory is nearly exhausted	the top memory processes are in the finding; a leak, or undersized
63% of swap is in use	the host is short of memory	as above
CPUs spend 35% of their time waiting for disk	slow or overloaded storage	<code>iostat -x</code> shows which device; storage latency
the hypervisor takes 15% of CPU time (steal)	the VM host is overcommitted	the virtualization team
the kernel killed processes for lack of memory	OOM kills in the last 24 h (with the victims)	the victim's memory limit, or the host's memory
27 zombie processes	a parent process is not reaping its children	restart the parent (<code>look</code> : shows it)

time

Finding	Means	Usually
the clock is NOT synchronized	chrony uses no time source	<code>chronyc sources -v</code> : unreachable servers, firewall (UDP 123)
chrony is installed but not answering	chronyd is not running	<code>systemctl start chronyd</code>
the clock is 2.4s ahead of NTP time	offset above <code>check_time_offset_warn</code>	chrony corrects it slowly; very large offsets break Kerberos/AD logins at 5 minutes

network

Finding	Means	Usually
there is no default route	the host cannot reach other networks	NetworkManager connection, gateway
this host's own name ... does not resolve	its FQDN is not in DNS or <code>/etc/hosts</code>	DNS record
ldap.example.mil does not resolve (DNS)	a name in <code>check_network_dns_names</code> fails	<code>/etc/resolv.conf</code> , DNS server
LDAP (ldap.example.mil:636) does not answer within 5s	a TCP port in <code>check_network_tcp</code> is unreachable from this host	firewall (local or network), the service down, routing

logging

Finding	Means	Usually
the journal is kept in memory only	<code>/var/log/journal</code> missing: logs vanish at reboot	<code>mkdir /var/log/journal && systemctl restart systemd-journald</code> (or <code>Storage=persistent</code>)
log server SIEM (...) does not answer	forwarding target unreachable	network/firewall; the SIEM collector
rsyslog has 850 MB of undelivered messages queued	the log server has been unreachable for a while	as above; the queue drains once it answers
1203 error messages in the journal	something complains a lot (top senders listed)	look at the top sender first

selinux

Finding	Means	Usually
SELinux is Permissive, it must be enforcing	not enforcing (STIG)	<code>setenforce 1</code> after checking denials; <code>SELINUX=enforcing</code> in <code>/etc/selinux/config</code>
<code>/etc/selinux/config</code> says <code>SELINUX=permissive</code>	a reboot would change the mode	fix the config file
locally added permissive domain(s)	SELinux only logs for those domains	<code>semanage permissive -d <domain></code> once the policy is right
12 SELinux denial(s) since yesterday - top: nginx (httpd_t) denied read on a file labeled user_home_t	SELinux blocked something	a wrong label (fix: <code>restorecon -Rv <path></code>), a boolean (<code>setsebool -P ...</code>), or a real policy need. <code>audit2why</code> says which
SELinux boolean X is off, expected on	a boolean in <code>check_selinux_booleans</code> differs	<code>setsebool -P X on</code>
N file(s) under <code>/etc/nginx</code> have the wrong SELinux label	a dry-run <code>restorecon</code> would relabel them (<code>check_selinux_relabel_paths</code>)	<code>restorecon -Rv <path></code>

fapolicyd

Finding	Means	Usually
fapolicyd is inactive	application allow-listing is off	<code>systemctl start fapolicyd</code> ; its log says why it stopped
fapolicyd is in permissive mode	it logs but blocks nothing	<code>permissive = 0</code> in <code>/etc/fapolicyd/fapolicyd.conf</code>
N file(s) no longer match the fapolicyd trust database	files changed outside RPM	<code>fapolicyd-cli --update</code> , or re-add the file's trust entry
N fapolicyd denial(s) since yesterday - top: /usr/bin/bash -> /opt/app/tool	it blocked programs	a legitimate program needs a trust entry (<code>fapolicyd-cli --file add <path></code>), after review

`check_fapolicyd_required: true` (the example inventory sets it) makes a missing fapolicyd a finding.

auditd

Finding	Means	Usually
auditd is failed	nothing is being audited	<code>systemctl status auditd</code> ; often a full audit log partition
auditing is DISABLED in the kernel	<code>auditctl -s</code> says enabled 0	<code>audit=1</code> on the kernel command line
only 1 audit rule(s) are loaded	the STIG rules are missing or failed to load	<code>augenrules --check</code> , <code>augenrules --load</code> ; a syntax error in <code>/etc/audit/rules.d/</code>
N audit event(s) were LOST	the kernel backlog overflowed	raise <code>-b</code> (backlog) in the rules
the audit log filesystem is 92% full	auditd may stop, or halt the host, when it fills	rotate/offload; check <code>space_left_action</code>
audit records are not forwarded	au-remote is not active (<code>check_auditd_remote_required</code>)	configure audisp-remote

accounts

Finding	Means	Usually
account(s) other than root have UID 0	full root power under another name	investigate first ; remove or renumber the account
account(s) with an EMPTY password	log in without a password	<code>passwd -l <user></code>
3 account(s) not used for 35+ days	inactive and not locked (STIG)	lock them (<code>usermod -L</code>), or list break-glass accounts in <code>check_accounts_ignore_inactive</code>
sudo without a password (NOPASSWD) for: alice	a sudoers entry skips the password	remove it, or allow it on purpose in <code>check_accounts_nopasswd_allowed</code> (e.g. <code>svc_aap</code>)
the password of svc_aap expires in 6 days	an account in <code>check_accounts_expiry_watch</code> expires soon	change it before it breaks automation (and update the Machine credential)

certs

Finding	Means	Usually
certificate CN=... in /etc/stigman/stigman.crt expires in 9 days	a certificate file expires within <code>check_certs_crit_days</code> (critical) / <code>warn_days</code> (warning)	renew it. For STIG Manager: update the <i>TLS certificate</i> credential and run <i>STIG Manager - deploy</i> ; it restarts only nginx
... in STIG Manager (https://127.0.0.1:443) expires in ...	the certificate a TLS endpoint serves	the service's certificate, even if the file on disk was already replaced (restart the service)
... in /opt/.../agent_keystore (alias mid) expires in ...	a Java keystore entry	renew in the keystore (MID Server: ServiceNow's procedure)
could not check the certificate in X	unreadable file, endpoint not answering, or a PKCS12 keystore without its password	fix the path, or add the Keystore password credential

What it looks at: `check_certs_files` (globs), the usual directories (`check_certs_discover_dirs`, skipping CA bundles), `check_certs_endpoints`, `check_certs_keystores`. For a PKCS12 keystore, name the variable that holds the password (`password_env: KEYSTORE_PASSWORD`), never the password.

patching

Finding	Means	Usually
12 security advisories pending, 2 of them rated Critical	updates that fix security issues are waiting	patch (the <i>Patch with checks</i> workflow)
a reboot is needed to finish earlier updates	new kernel or core libraries are not in use yet	reboot in a window
no package has been updated for 60 days	older than <code>check_patching_max_age_days</code>	patch
cannot check for updates	dnf cannot reach its repositories (Satellite, mirror)	repository configuration, network, subscription

mariadb (health only)

Runs on hosts in the `mariadb_hosts` group. It runs status queries only: no data is read and nothing is changed. For MariaDB in a container (the ServiceNow database), set `check_mariadb_container: <name>`; the client inside the container is used.

Finding	Means	Usually
MariaDB is not running	the service or container is down	its log; disk full; a crash
MariaDB is running but the status query failed	cannot log in, or it hangs	the login (see below); <code>max_connections</code> reached
MariaDB restarted 12 minutes ago	a restart nobody may have planned	the error log: crash, OOM kill
143 of 151 connections in use (95%)	clients are about to be refused	the <code>look</code> : query shows which user and client hold them; a connection leak in an app
N client connection(s) were REFUSED	<code>max_connections</code> was reached since the last start	as above; raise <code>max_connections</code> only if the load is real
40 queries are executing at this moment	a pile-up: locking or one slow query	<code>look</code> : lists the oldest
2 query/queries running longer than 300s	long-running queries (user / db / seconds / state; never the query text)	the owner decides; <code>KILL <id></code> only after that
replication (default) is broken: IO thread No	a replica stopped copying	<code>Last_IO_Error</code> / <code>Last_SQL_Error</code> in the finding
replica ... is 1900 seconds behind	replication lag	load on the replica; long transactions on the primary
Galera node is not healthy / cluster has 2 node(s), expected 3	Galera cluster trouble	the node's error log; network between nodes
only 91% of reads come from memory	<code>innodb_buffer_pool_size</code> may be too small	the DBA

Finding	Means	Usually
3 [ERROR] line(s) in the MariaDB log	errors in its log in the last day (latest shown)	read them
the MariaDB data directory is on a filesystem 93% full	MariaDB stops writing when it fills	disk space

Logging in. Without a credential, the check logs in as root through the local socket. That is MariaDB's default for a host install on RHEL (no password needed with sudo). **MariaDB in a container usually gives root a password** (the official images do), and the check then reports `Access denied ... attach the "MariaDB monitor" credential`. For containers, or whenever you prefer not to use root, ask the DBA to create a monitoring account with **no data access**, inside that database with host `localhost`, and attach the *MariaDB monitor* credential to the Health check template:

SQL

```
CREATE USER 'aap_monitor'@'localhost' IDENTIFIED BY '<password>';
GRANT PROCESS, SLAVE MONITOR ON *.* TO 'aap_monitor'@'localhost'; -- MariaDB 10.5.9+
-- older MariaDB: GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';
```

`PROCESS` shows other sessions (for long queries); `SLAVE MONITOR` shows replication status. The password reaches the client only through its environment, never its command line.

Troubleshoot (playbooks/troubleshoot.yml)

For "something is wrong on this host": pick the area and the job runs the commands an experienced admin runs first. Each is printed under a line saying what it shows, and the matching health checks run too. Read-only. Findings do not fail this job: you asked for a look, not a verdict.

Area	Commands (what they show)	Checks
overview	uptime, failed units, disk, memory, recent errors, recent reboots	disk, mounts, services, performance, time, logging
disk	space and inodes, biggest directories on the fullest filesystem, deleted-but-open files, files over 1 GiB, journal size	disk, mounts
performance	load, top, vmstat, memory users, disk latency (iostat), load history (sar), OOM kills	performance
service (<code>ts_service</code>)	its status, why it stopped (result, exit code, restarts), its log, dependencies, unit file, listening ports	services (plus that unit)
network (<code>ts_target host:port</code>)	addresses, routes, DNS servers, name lookup, TCP connect, route taken, firewall, connection counts	network (plus that target)
selinux	mode, today's denials, audit2why, changed booleans, local file labels	selinux
fapolicyd	status, today's denials, rules, enforcing?	fapolicyd
login	sshd log, faillock lockouts, failed logins, SSSD domain status, authselect, clock offset	accounts

Area	Commands (what they show)	Checks
time	chrony tracking and sources, timedatectl, configured servers	time
logs	errors, top error senders, rsyslog and its queue, journal size	logging
mariadb	(the MariaDB health check)	mariadb

Add `use_act=true` and ACT reads everything collected and explains the root cause (ADDING_ACT.md).

Certificate report (playbooks/cert_report.yml)

The certs check on every host, and **one table**, soonest expiry first: days left, date, host, subject, and where the certificate is. Hand it to whoever renews certificates. It fails the hosts with a certificate inside `check_certs_warn_days`, so a workflow can open tickets.

POA&M status (playbooks/poam_status.yml)

Reads `poam/poam.csv` (see `poam/README.md` for the columns and how to export from eMASS):

Finding	Means
POA&M 1001 [CAT II]: ... - OVERDUE by 12 days	open item past its scheduled completion date (critical)
POA&M 1002 ...: due in 20 days	due within <code>poam_due_soon_days</code> (30)
... scheduled completion date "TBD" is missing or not a date	fix the date in the list
open CAT I finding V-123456 (...) on 2 asset(s) in collection Production has no POA&M item	STIG Manager cross-check: an open CAT I/II finding that no POA&M item covers

It also lists "maybe ready to close": items whose STIG findings are no longer open in STIG Manager.

The STIG Manager cross-check is optional. It needs a Keycloak client that may use the client-credentials grant, with the scope `stig-manager:collection:read`, and a grant in the collections it reads. **Your realm owner creates that client**; nothing here changes the realm. Then create the *STIG Manager API* credential (token URL `https://<keycloak>/realms/<realm>/protocol/openid-connect/token`, client ID, secret), and set `poam_stigman_api: https://<stigman>/api`.

ServiceNow tickets (playbooks/servicenow_tickets.yml)

A workflow step after a check. For each finding at or above `servicenow_min_severity`:

- **no open ticket** for that problem on that host › it opens an incident: short description `[site-health] host: summary`; description with the host, check, finding, the `look:` command, and ACT's analysis if ACT ran; urgency/impact from the severity; assignment group, category and configuration item from the settings.
- **an open ticket exists** › it adds a work note `Still failing at ... (servicenow_note_repeats)`.
- **a host that was targeted but never reported** › a ticket too (unreachable, login failed).

- **a problem that cleared** (its check ran again and passed) › a work note on its ticket, or it resolves it with `servicenow_resolve_cleared: true`.

Tickets are matched by a correlation ID (`site-health:<host>:<check>:<finding>`, hashed), never by text, so the same problem always updates the same ticket. Without the ServiceNow API credential it only prints what it would open. If no check results reach it at all (the check step died before checking any host), it fails, so a broken schedule shows red instead of green.

ServiceNow health (playbooks/servicenow_health.yml)

From the controller: does the instance answer its REST API, and how fast (`servicenow_slow_seconds`)? Is every MID Server **Up** and **validated**? Hourly is a good schedule. Alert with an AAP **notification** (email) on failure, not with a ticket: a broken ServiceNow cannot take tickets. The MID Server **hosts** (the Linux service, its keystore) are covered by the health check: see `inventories/example/group_vars/servicenow_mid_hosts.yml`.

Patch hosts (playbooks/patch_hosts.yml)

Installs updates with dnf (`patch_security_only` for security fixes only; `patch_exclude` for packages never updated here). Then:

1. **One host at a time** (`patch_serial`). The rollout **stops at the first host that fails**.
2. Refuses with less than `patch_min_free_mb` free in `/var`.
3. **No automatic restarts** unless `automatic_restarts: true` (the survey or the template's Variables). Without it, a host that needs a reboot is listed ("reboot it by hand, or run again with `automatic_restarts: true`"). With it, the job reboots only when `needs-restarting -r` says so (`patch_reboot: when_needed`), and **never** hosts in `patch_never_reboot_groups` (`aap_hosts`).
4. After the update (and reboot), **every service that was running before must be running again**. If one is not, the host fails and the rollout stops.
5. Skips hosts in `no_patch` (vendor appliances, and AAP itself in the example inventory), and refuses to patch the machine running the job.

Run it as **Job type: Check** first: it lists what would be updated and changes nothing (DRY_RUNS.md).

Apply approved ACT fix (playbooks/act_fix_approved.yml)

The step after an approval in the *Fix with approval (ACT)* workflow. It runs **exactly** the commands ACT proposed and a person approved, then re-runs the checks that found the problem. It refuses to run on its own, and skips diagnose-only hosts (`aap_hosts`, `netapp_console_hosts`): there a person applies the fix by hand. See `ADDING_ACT.md`.

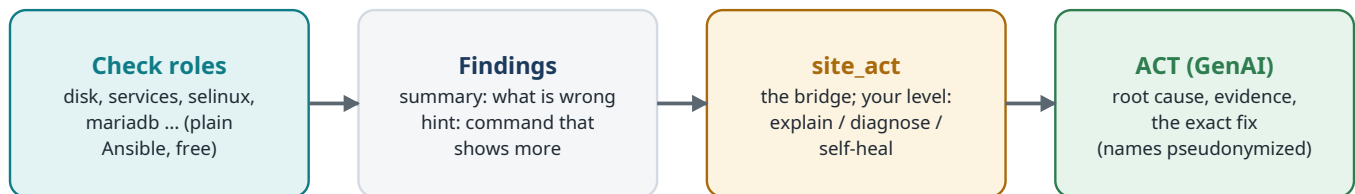
Not built yet (and why)

Idea	Why not yet
SCAP scan › import into STIG Manager	Needs the same Keycloak client as the POA&M cross-check, plus a decision on SCC vs OpenSCAP content. STIG Manager's own importer (stigman-watcher) is the supported path
Certificate renewal workflow	Depends on how your PKI issues certificates. For STIG Manager, renewal already works: update the TLS credential, run <i>STIG Manager - deploy</i>
AAP self-health (job failure rate, execution environment and token age)	Needs an AAP API credential; AAP's own dashboards cover most of it today
Container runbooks (image updates with rollback, drift)	Deferred on purpose; <i>Service watch</i> covers container outages
MariaDB backups, security baseline, password rotation	Out of scope for now: health only, as asked

7. Adding ACT (GenAI) later

The runbooks work without ACT. Add it when the checks are running well and you want the next step done for you: **why** it broke and **what** fixes it. This page shows how the runbooks and ACT fit together, how to turn it on per template, and how to write your own check so it works with ACT automatically.

How they fit: checks find WHAT, ACT finds WHY



Report, ServiceNow tickets and the approval workflow read the same findings - with or without ACT.

Checks find WHAT is wrong; ACT finds WHY. A new check works with ACT automatically through its findings' hints.

- The **checks** decide *whether* something is wrong. They are plain Ansible: the same answer every time, and they are free. The model is only called for hosts with findings.
- Every finding carries a **hint**: the read-only command a person would run next. ACT gets those commands' output as evidence. A new check you write is therefore usable by ACT with no extra work.
- `roles/site_act` is the only place that talks to ACT. Every runbook uses it the same way. Turning ACT on is a variable, not a code change.
- Host names, IP addresses and user names are **pseudonymized** before anything reaches the model (ACT 0.6.18). Its replies are translated back on the host.

The three levels

Level	ACT does	Changes to hosts	Approval step?
explain	reads the evidence (the hint commands' output, collected by Ansible) and writes the root cause and a suggested fix. It runs no commands itself	none	no
diagnose	investigates by running read-only commands itself, then proposes exact fix commands. Every change is refused and waits for a person	only after approval	yes: the job fails with NEEDS APPROVAL › approval step › <i>Apply approved ACT fix</i>
self-heal	like diagnose , but may run fixes that match <code>site_act_allow</code> by itself (for example <code>systemctl restart chronyd</code>). Anything else still waits for approval. The checks then run again to prove the fix worked	only fixes you listed	only for the other fixes

Start with **explain**. Move a runbook to **diagnose** when ACT's explanations have been right for a while. Use **self-heal** only for fixes you would let a junior admin do unasked: start a stopped service, restart a hung

one.

Never fixed automatically, at any level: hosts in `site_act_diagnose_only_groups` (default: `aap_hosts`, `netapp_console_hosts`). There `self-heal` becomes `diagnose`, and the apply step skips them: it prints the approved command for a person to run by hand.

Turn it on

1. The ACT model key credential

Create the credential type `ACT model key` (`aap/credential_types/act_model_key.yml`) and one credential with your key: the GenAI key for GenAI.mil, or the AskSage key for Ask Sage (SETUP_AAP.md, steps 2-3). For the GenAI beta proxy, also create `ACT GenAI beta key` (`act_genai_beta_key.yml`). Attach the credential(s) to the **Health check** and **Troubleshoot** templates (and *Service watch - check*), next to `Linux ssh (sudo)`. This is the only place a key lives. The apply steps do not call the model: they run the approved commands themselves, so they need no key.

2. Pick the model provider

ACT can talk to three providers. Pick one with `site_act_provider`:

site_act_provider	Provider	Key (credential)	Also set
<code>genai</code> (default)	GenAI.mil	<i>ACT model key</i> , GenAI API key field	nothing (optional: <code>site_act_model</code>)
<code>asksage</code>	Ask Sage	<i>ACT model key</i> , AskSage API key field	<code>site_act_url</code> : your organization's Ask Sage API URL. The default is <code>https://api.genai.army.mil/server/openai/v1/chat/completions</code>
<code>genai-beta</code>	GenAI.mil beta proxy (preview models)	<i>ACT GenAI beta key</i> (<code>aap/credential_types/act_genai_beta_key.yml</code>)	nothing

The four settings:

Setting	What	Default
<code>site_act_provider</code>	<code>genai</code> , <code>asksage</code> or <code>genai-beta</code>	<code>genai</code>
<code>site_act_models</code>	the model for each provider , so the model follows the provider you pick, e.g. <code>{genai: gemini-3.8-flash, asksage: gpt-5.6-sol-gov}</code>	ACT's default for the provider
<code>site_act_urls</code>	the API URL for each provider, e.g. <code>{asksage: https://<host>/server/openai/v1/chat/completions}</code>	ACT's default for the provider
<code>site_act_model</code> , <code>site_act_url</code>	one model / URL for this run, whatever the provider (a survey answer)	empty: use the two above
<code>site_act_ca</code>	a CA bundle file on the hosts , if the provider's certificate comes from a CA the hosts do not trust yet (for example a DoD CA not in <code>/etc/pki/ca-trust</code>)	the host's normal trust store

Where to set them:

- **For everyone:** `playbooks/group_vars/all.yml`, already set up like this. Not the inventory's Variables box in AAP: this file beats it, so a value there would be ignored.

YAML

```
site_act_provider: genai          # the default provider
site_act_models:                 # the model follows the provider you pick
  genai: gemini-3.8-flash
  asksage: gpt-5.6-sol-gov
site_act_urls:                  # only if your Ask Sage is not ACT's default URL
  asksage: https://<your ask sage host>/server/openai/v1/chat/completions
```

To make Ask Sage the default, change the first line to `site_act_provider: asksage`.

- **For one group:** the group's settings file (`playbooks/group_vars/<group>.yml`), or the group's **Variables** box in AAP. For example `site_act_provider: asksage` there, and only that group's hosts use Ask Sage (with its model from `site_act_models`).
- **For one template:** its **Variables** box (*Extra variables*), for example `site_act_provider: asksage`. It beats everything above, for every run of that template.
- **Per launch:** the survey questions in step 3 (a survey answer beats everything too).

The API **key** is never a variable: it comes from the *ACT model key* credential on the template.

These settings apply to every runbook that uses ACT, including *Service watch* and *Apply approved ACT fix*. If the key for the chosen provider is missing, ACT does not run: the job says `ACT did not run: this job has no API key for provider asksage. Attach ...`, and the findings are still reported.

Network: ACT runs **on each host**, not in AAP. So each host must be able to reach the provider's URL over HTTPS. If they go through a proxy, add it: `site_act_env: {HTTPS_PROXY: "http://proxy.yoursite.mil:8080"}`.

Anything else ACT should know about your site goes in the inventory too:

YAML

```
site_act_extra_instructions: >-
  RHEL 9 servers under DISA STIG. fapolicyd and SELinux are enforcing. Never suggest disabling
  them.
```

3. Survey questions

On **Health check** and **Troubleshoot** › **Survey** › **Create survey question:**

Question	Variable	Type	Choices	Default
Ask ACT (GenAI) about the findings?	<code>use_act</code>	Multiple Choice (single select)	<code>no, yes</code>	<code>no</code>
What may ACT do?	<code>site_act_level</code>	Multiple Choice (single select)	<code>explain, diagnose, self-heal</code>	<code>explain</code>
Which model provider? (optional)	<code>site_act_provider</code>	Multiple Choice (single select)	<code>genai, asksage, genai-beta</code>	the one in your inventory

Question	Variable	Type	Choices	Default
Which model? (optional, blank = the provider's model from <code>site_act_models</code>)	<code>site_act_model</code>	Text, not required		blank

Leave out the last two if everyone uses the same provider. The inventory setting then applies.

4. Try explain by hand

Launch **Troubleshoot** on a host with a known problem: `ts_area = service`, `ts_service = the unit`, `use_act = yes`, `site_act_level = explain`. In the output, read `ACT | what ACT says`. With ACT on, the host's ServiceNow ticket includes this analysis too.

5. diagnose with the approval workflow

Build *Fix with approval (ACT)* (WORKFLOWS_AND_SCHEDULES.md). What happens:

1. The health check finds, for example, `required service chronyd.service is inactive`.
2. ACT (`diagnose`) looks at the unit, its log and the config, and proposes `systemctl start chronyd`. The command is refused, so the job fails: `NEEDS APPROVAL - ... ACT proposes: systemctl start chronyd`.
3. The workflow stops at the approval step. The approver reads ACT's report and approves.
4. **Apply approved ACT fix** lets ACT run **exactly** `systemctl start chronyd` (the approved text, escaped, is the only pattern allowed), and ACT verifies it.
5. The same checks run again, without ACT, and the job is green only if the problem is gone. The tickets step notes the ticket as cleared.

6. self-heal for the fixes you trust

```
inventories/site/group_vars/all.yml

site_act_allow:
  - 'systemctl (start|restart) (chronyd|rsyslog|crond)'
```

Each entry is a regular expression that must match the **whole** command. Keep them narrow: name the services, never `.*`. Then pick `site_act_level = self-heal`. After ACT applies an allowed fix, the checks with findings run again. The report shows `cleared: ...` or `still there: ...` and reflects the state **after** the fix.

Write your own check (it works with the report, tickets and ACT automatically)

A check is a role named `check_<name>` that **adds findings** to `site_findings`. That is the whole contract. As an example, here is a check that `/tmp` is not world-writable without the sticky bit.

1. Create `roles/check_tmp/defaults/main.yml`, with its settings and a comment saying what it checks:

```
YAML

---
# TMP: /tmp and /var/tmp must have the sticky bit (mode 1777).
check_tmp_paths: [/tmp, /var/tmp]
```

2. Create `roles/check_tmp/tasks/main.yml`. Collect with read-only commands, and always use `changed_when: false` and `check_mode: false`: read commands then run in a dry run too. Turn what you saw into findings, and add them:

YAML

```

---
- name: tmp | modes
  ansible.builtin.command:
    argv: "{{ ['stat', '-c', '%a %n'] + check_tmp_paths }}"
  register: _tmp
  changed_when: false
  failed_when: false
  check_mode: false

- name: tmp | findings
  ansible.builtin.set_fact:
    _tmp_findings: >-
      {%- set f = [] -%}
      {%- for line in _tmp.stdout_lines if line.split()[0] != '1777' -%}
      {%-   set path = line.split()[1] -%}
      {%-   set _ = f.append({'check': 'tmp', 'id': 'tmp:' ~ path, 'severity': 'warning',
                            'summary': path ~ ' has mode ' ~ line.split()[0] ~ ' (expected
                            1777)',
                            'hint': 'stat ' ~ path ~ '; findmnt ' ~ path}) -%}

      {%- endfor -%}
      {{ f }}

- name: tmp | add them to this host's findings
  ansible.builtin.set_fact:
    site_findings: "{{ site_findings | default([]) + _tmp_findings }}"

```

- Let the health check know it: in `playbooks/health_check.yml`, add `tmp` to `_health_sets` (to `weekly` and `all`), and add `tmp` to the survey choices of the Health check template.
- Test it on one host: `ansible-playbook playbooks/health_check.yml -l <host> -e health_checks=tmp`. Then break the condition on purpose (`chmod 777 /tmp` on a test host) and check that the finding appears. A check that never fires proves nothing.

That is all. The report prints it, `set_stats` publishes it, ServiceNow tickets it (the `id` keeps it on one ticket), and with `use_act=yes` ACT receives the summary and runs the `hint` as evidence.

Rules for a good finding:

- summary**: one line a person understands, with the number and the limit (`/var is 96% full (critical at 95%)`).
- id**: stable across runs, with no numbers that change (`disk:/var`, not `disk:/var:96`). Otherwise every run opens a new ticket.
- hint**: **read-only**, runs in under a minute (`timeout 60 ...`), and shows the cause, not just the symptom. ACT runs it as root in `explain` mode, so never put a command that changes something there.
- severity**: `critical` = act now (outage, security); `warning` = act soon.

Use ACT in a playbook of your own

Any playbook that fills `site_findings` can hand them to ACT the same way:

YAML

```
- name: My check
  hosts: my_hosts
  become: true
  gather_facts: false
  tasks:
    - ansible.builtin.include_role: {name: site_findings, tasks_from: start.yml}
    - ansible.builtin.include_role: {name: site_findings, tasks_from: run_check.yml}
      vars: {_site_check: tmp}                                # runs roles/check_tmp
    - ansible.builtin.include_role: {name: site_act}
      when: use_act | default(false) | bool and site_findings | length > 0
    - ansible.builtin.include_role: {name: site_findings, tasks_from: report.yml}
```

`report.yml` goes last. It prints, publishes (for tickets and the approval workflow), and fails the host when there are findings or a fix waits for approval.

What ACT is told, and what it cannot do

- Its task: the findings (one line each), "find WHY, confirm it with evidence, give the exact fix; several findings may share one cause", plus your `site_act_extra_instructions`.
- `explain`: it runs nothing. Ansible ran the hint commands.
- `diagnose`: only commands ACT can prove are read-only run on their own. Everything else is refused and becomes a proposal.
- `self-heal`: in addition, only commands matching `site_act_allow`.
- Apply step: no model at all. The playbook runs only the approved commands, character for character, in order, and stops at the first one that fails. Run as *Check*, it only shows what it would run (a dry run: see WORKFLOWS_AND_SCHEDULES.md, "Rehearse it").
- Every run is recorded in the AAP job output. The ticket carries ACT's analysis. For the service-watch style incident log, see SERVICE_WATCH_DEMO.md.

8. Windows servers

The Windows runbooks work the way the Linux ones do. Checks find what is wrong and say it in plain words, with a command to look further. The same report, the same ServiceNow tickets and POA&M, and the same workflows and schedules follow. Only the checks themselves are different: each one is a PowerShell script that runs on the server over WinRM.

Nothing extra to install on AAP or on the servers. The playbooks use only Ansible's built-in `script` module, so no Ansible collection is needed. The execution environment needs the Python library for WinRM (`pywinrm`); step 2 shows how to check. The servers need WinRM over HTTPS, which you already use. The check scripts run in Windows PowerShell 5.1 and PowerShell 7, and also in Constrained Language Mode (see step 5).

Template	Playbook	What it does
Windows connection test	<code>playbooks/win_connection_test.yml</code>	run it first: can AAP reach the servers, with the right account?
Windows health check	<code>playbooks/win_health_check.yml</code>	the checks below; daily and weekly sets
Windows troubleshoot	<code>playbooks/win_troubleshoot.yml</code>	on demand: an admin's first PowerShell commands for one kind of problem
Windows certificate report	<code>playbooks/win_cert_report.yml</code>	every certificate on every server, soonest expiry first
Windows patch	<code>playbooks/win_patch.yml</code>	install what Software Center offers (ConfigMgr); restart only with <code>automatic_restarts: yes</code> ; prove the server is back
ACT for Windows - install	<code>playbooks/win_act_install.yml</code>	put ACT on the servers in <code>C:\ProgramData\act</code> , locked down and unblocked; keep it current
ServiceNow tickets, POA&M status	the ones you have	unchanged: they take the Windows results the same way

The checks (`roles/win_check_<name>`):

Set	Check	Finds
daily	<code>disk</code>	volumes too full
daily	<code>services</code>	required services not running; automatic services that stopped with an error; services that keep crashing
daily	<code>performance</code>	CPU, memory or page file too busy; names the busiest processes
daily	<code>time</code>	Windows Time not running, not synchronized, synced too long ago, clock off
daily	<code>network</code>	no gateway or DNS; names that do not resolve; ports that do not answer; a broken domain trust

Set	Check	Finds
daily	<code>eventlog</code>	unexpected shutdowns, crashes, disk errors, a cleared Security log, many errors, full logs
weekly	<code>security</code>	Defender real-time protection off or old signatures; endpoint services down; firewall off; SMBv1; RDP without NLA; UAC off; TLS 1.0/1.1
weekly	<code>audit</code>	audit policy settings the STIG asks for; event log sizes; the log forwarding agent
weekly	<code>accounts</code>	Guest enabled; local passwords old or never expiring; inactive local accounts; unexpected local Administrators
weekly	<code>certs</code>	certificates that expire or expired, including the WinRM listener AAP connects through
weekly	<code>patching</code>	a restart pending for installed updates; days since the last update

Everything is read-only. The one write is one Application event log entry per finding (source `site-health`), so a SIEM that collects event logs sees the findings too.

Step 1. WinRM over HTTPS on the servers

You already use it. To see it on a server:

```
winrm enumerate winrm/config/listener
```

It shows a listener with `Transport = HTTPS`, `Port = 5986` and a `CertificateThumbprint`. The firewall must let port 5986 in from the AAP execution nodes. The `certs` check warns before that listener's certificate expires: when it expires, AAP can no longer connect.

Step 2. The execution environment

The job runs inside an execution environment (EE), and that EE needs these Python libraries:

- `pywinrm`, always;
- `requests-ntlm`, for NTLM (the usual choice);
- or `requests-kerberos` plus a Kerberos client (`kinit`), for Kerberos.

AAP's *Default execution environment* has them. Your own EE (for example `platforms-ee`) may not. **The Windows connection test tells you:** its first part runs inside the EE and prints `YES` or `no` for each library. If `pywinrm` is missing, you have two options:

- use AAP's Default execution environment for the Windows templates; or
- add the libraries to your EE's definition and rebuild it:

```
YAML
dependencies:
  python:
    - pywinrm
    - requests-ntlm
```

Step 3. The credential

Automation Execution › Infrastructure › Credentials › Create credential:

- **Name:** `Windows (WinRM)`; **Credential type:** `Machine`.
- **Username:** `YOURDOMAIN\svc_aap` for NTLM. For Kerberos, use `svc_aap@YOURDOMAIN.MIL`, with the domain in capitals.
- **Password:** the account's password.
- **Privilege escalation:** leave it empty. An administrator's WinRM session is already elevated.

The account must be in the **local Administrators group** on every server, usually through a domain group added by GPO. The checks need that to read the Security log, the audit policy, local accounts and certificate stores.

Step 4. The Windows inventory

Keep the Windows servers in their **own inventory**. Go to **Inventories › Create inventory**, name it `Windows servers`, and paste this in its **Variables** box:

```
YAML
ansible_connection: winrm
ansible_port: 5986
ansible_winrm_scheme: https
ansible_winrm_transport: ntlm          # or kerberos
ansible_winrm_server_cert_validation: ignore # see below
```

- **Certificate validation.** `validate` means the EE checks the server's WinRM certificate against the CAs it trusts. EEs usually do not include the DoD root CAs, so `validate` fails with `CERTIFICATE_VERIFY_FAILED`. `ignore` keeps the traffic encrypted but does not check who the server is. To validate, add your CA bundle to the EE and set `ansible_winrm_server_cert_validation: validate` and `ansible_winrm_ca_trust_path: /etc/pki/ca-trust/source/anchors/dod.pem` (the path inside the EE).
- **Hosts.** Add them by full name (`web01.yoursite.mil`); Kerberos needs the full name.
- **Groups.** Use whatever helps: by site, by role (`win_dc`, `win_web...`). The settings below can be set per group. `inventories/example-windows/hosts.yml` shows an example.

Linux and Windows do not mix: the Linux templates skip Windows servers, and the Windows templates skip Linux servers. So a template pointed at the wrong inventory changes nothing and opens no tickets.

Step 5. Test the connection

Create a job template, then launch it with **Limit** set to one server:

Field	Value
Name	<code>Windows connection test</code>
Inventory	<code>Windows servers</code>
Project	<code>site-automation</code>
Playbook	<code>playbooks/win_connection_test.yml</code>
Execution environment	one with <code>pywinrm</code> (step 2)

Field	Value
Credentials	Windows (WinRM)
Limit	empty, Prompt on launch ticked

A good result ends with:

```
Connected: WEB01 (Microsoft Windows Server 2022 Standard, build 20348), domain yoursite.mil
PowerShell 5.1.20348.2849, FullLanguage
Logged on as yoursite\svc_aap; local administrator: yes
OK: the Windows runbooks can run here.
```

ConstrainedLanguage instead of **FullLanguage** means WDAC or AppLocker restricts PowerShell on that server. The check scripts are written and tested for that mode; only two optional parts are skipped in it (TLS endpoints and the search for missing updates). Whether Ansible itself can run on such a server depends on your ansible-core version and the policy: if this test got through and printed its result, it can.

Step 6. The job templates

For all of these, use: **Inventory** **Windows servers**, **Project** **site-automation**, the EE from step 2, **Credentials** **Windows (WinRM)**, **Limit** empty with **Prompt on launch** ticked. Leave **Variables** empty: with no **target**, they run on every server in the inventory.

Windows health check

Field	Value
Playbook	playbooks/win_health_check.yml
Survey: health_checks	<i>Multiple Choice (multiple select): daily, weekly, all, disk, services, performance, time, network, eventlog, security, audit, accounts, certs, patching. Default daily</i>

Windows troubleshoot

Field	Value
Playbook	playbooks/win_troubleshoot.yml
Survey: ts_area	<i>overview, disk, performance, service, network, login, time, logs, updates, security. Default overview</i>
Survey: ts_service	<i>Text, not required. "Service name (for area = service), e.g. W3SVC"</i>
Survey: ts_target	<i>Text, not required. "host:port it cannot reach (for area = network)"</i>
Survey: ts_hours	<i>Integer, default 2. "How many hours of event logs to read"</i>

Each command's output appears under a **### what it shows** line, followed by the matching health checks. Findings do not fail this job.

Windows certificate report

Field	Value
Playbook	<code>playbooks/win_cert_report.yml</code>
Survey	none

Step 7. Workflows and schedules

These are the same as for Linux, with the Windows templates. The full list, with every step and link, is in `WORKFLOWS_AND_SCHEDULES.md` (workflows 12 to 17):

Workflow	Steps	Schedule
Windows daily health	Windows health check (daily) › ServiceNow tickets (Run always)	every day 06:15
Windows weekly compliance	Windows health check (weekly) › tickets; Windows certificate report › its own tickets box (both Run always)	Monday 07:15
Windows patch with a dry run first	Windows health check (disk, services) › Windows patch as Check › <i>Approval</i> › Windows patch › Windows health check (daily) › tickets	monthly, inside the servers' ConfigMgr maintenance window

The ServiceNow tickets template runs on AAP itself, so it stays as it is.

Start small: the pilot order

Everything Windows here is new, so try it on **one** server that does not matter much, in this order, and widen only when each step looks right:

1. **Windows connection test**: the account, WinRM and PowerShell work.
2. **Windows health check**: the findings make sense for that server.
3. **Windows patch as Check** (the dry run): it lists what Software Center would install.
4. **Windows patch for real**, inside that server's ConfigMgr maintenance window.
5. **ACT for Windows - install** on the same server (tell your security team first, see below).

Every step can be a dry run first: `DRY_RUNS.md`.

Patching through Software Center

`playbooks/win_patch.yml` does, on each server, what an admin does in Software Center: install every software update your ConfigMgr admins deployed to it ("Install all"), required and available alike. AAP decides nothing about *which* updates: ConfigMgr does, as it does today. Applications listed in Software Center are not installed; only software updates are. Then:

1. **No automatic restarts** unless you ask for them with `automatic_restarts: yes` (the survey, or the template's Variables). Without it, the job installs the updates and lists the servers that need a restart: **A RESTART IS NEEDED to finish the updates ... automatic restarts are off**. A person restarts them (the post-check's `patching` finding keeps them visible). With it, the job restarts a server only when the updates or the ConfigMgr client say so, and never one in `win_patch_never_reboot_groups`.

2. **Prove the server is back** (after a restart, and also when none was needed). A new boot time, and every automatic service that ran before runs again. Delayed services get `win_patch_services_grace_min` (5 minutes) to start.
3. **Look again.** After a restart, servicing-stack and cumulative updates often reveal more updates. There is a second round (`win_patch_max_rounds: 2`), only when it restarted.

Safety.

- One server at a time, and the rollout stops at the first server that fails.
- By default it only patches the group `win_patch_hosts`. Set `target:` in the template's Variables (for example `target: all`) and pick the servers with the Limit at launch.
- Servers in `win_patch_never_patch_groups` (default `no_patch`) are skipped.
- **Maintenance windows still apply.** Outside the server's ConfigMgr maintenance window, ConfigMgr holds the updates. The job then stops with `ConfigMgr will not install these now ... (waiting for a maintenance window)` instead of waiting forever. Run it inside the window, or ask the ConfigMgr admin for a window that covers the patch time.
- **A dry run first:** run the template as *Check*. It rescans and lists what it would install, and changes nothing (DRY_RUNS.md). The *Windows patch with a dry run first* workflow puts that list in front of the approver.

Needs: the ConfigMgr client (Software Center) on the servers, and full PowerShell (not Constrained Language Mode) for the install call. Restarts use `shutdown.exe`, and AAP watches the WinRM port go down and come back. No Ansible collection is needed.

Scheduling. With `target: all` and no Limit, a scheduled run patches every server in the Windows inventory, one after another. Choose that on purpose. Otherwise, make one schedule per group (for example the test servers the week before the others), each with its own Limit.

The job template:

Field	Value
Name	<code>Windows patch</code>
Playbook	<code>playbooks/win_patch.yml</code>
Credentials	<code>Windows (WinRM)</code>
Job type	Run, with Prompt on launch ticked (Check = the dry run)
Variables	<code>target: all</code> (or a group), and pick servers with the Limit
Survey: <code>automatic_restarts</code>	"Restart servers automatically when the updates need it?" Choices <code>no</code> , <code>yes</code> . Default <code>no</code>

If a server fails:

What you see	What to do
<code>has no working ConfigMgr client</code>	the SMS Agent Host service (CcmExec) is missing or stopped
<code>waiting for a maintenance window</code>	run inside the window, or ask the ConfigMgr admin

What you see	What to do
these updates failed: ... (error 0x...)	C:\Windows\CCM\Logs\UpdatesDeployment.log and WUAHandler.log on the server
still not done after 120 minutes	Software Center on the server; C:\Windows\CCM\Logs\UpdatesHandler.log
did not accept a logon after the restart	look at the server's console
... ran before patching and do not run now	the service did not come back: <code>Get-Service <name></code> , System log

ACT on the servers (C:\ProgramData\act)

playbooks/win_act_install.yml puts ACT for Windows (vendor/act-windows/act.ps1) on the servers in C:\ProgramData\act (setting win_act_dir). Run it again after you update this repository, and every server gets the new version.

- **Locked down.** Ordinary users may create folders and files in C:\ProgramData (and in C:\temp). Someone could then place, or swap, act.ps1 before an administrator runs it, and their code would run with administrator rights. So the job:
 - takes ownership of C:\ProgramData\act;
 - lets only Administrators and SYSTEM in (nothing inherited from C:\ProgramData);
 - checks the file's SHA256 against the one in the repository, every run, and replaces it if it differs.
- **Unblocked.** The file is written on the server itself, so Windows never marks it as "downloaded from the internet", and the job runs Unblock-File on it anyway. Every run also checks it is not blocked, and fixes it if it is.
- **Using it on a server:** an administrator opens PowerShell and runs powershell -ExecutionPolicy Bypass -File C:\ProgramData\act\act.ps1. The first time, :setup asks for their own GenAI or Ask Sage key and keeps it in their profile. No key is put on the server for everyone.
- **Dry run:** run as Check and it says what it would install or fix. To remove it: win_act_state: absent.
- **Tell your security team before the first run.** The installer is a large PowerShell script that writes a .ps1 file, and some endpoint-protection products (Defender for Endpoint, Trellix) flag that pattern. Let them know it is coming from AAP, and try one server first.
- ACT itself needs full PowerShell. Where WDAC or AppLocker enforce Constrained Language Mode, ACT's own notes (sign it with your code-signing certificate) apply.

Settings: what each check looks at, and how to change it

Every setting is in roles/win_check_<name>/defaults/main.yml, with a comment saying what it does. To change one:

- **for all Windows servers,** use the Windows servers inventory's **Variables** box;
- **for a group,** use the group's **Variables** box, or a settings file named after the group (playbooks/group_vars/win_web.yml);
- **for one server,** use the host's **Variables** box.

The settings you will most likely want:

```
network: what every Windows server must resolve and reach
win_check_network_dns_names: [dc01.yoursite.mil]
win_check_network_tcp:
  - {name: LDAPS, host: dc01.yoursite.mil, port: 636}
  - {name: SIEM, host: siem.yoursite.mil, port: 9997}

# security: your endpoint protection's services (for example Trellix / McAfee)
win_check_security_av_services: [masvc, mfemms]

# audit: your log forwarding agent
win_check_audit_forwarder_services: [SplunkForwarder]

# accounts: who may be a local administrator (wildcards allowed)
win_check_accounts_admins_allowed: ['*\Administrator', 'YOURDOMAIN\Domain Admins',
  'YOURDOMAIN\svc_aap']

# services: more services that must run, on a group (for example IIS servers)
win_check_services_required_extra: [W3SVC, WAS]
```

A few defaults worth knowing:

- **disk:** warning at 85% used, critical at 95%. Per volume: `win_check_disk_overrides: {'D:': {warn: 92, crit: 98}}`.
- **services:** `EventLog`, `RpcSs`, `Dnscache`, `W32Time`, `WinRM` and `mpssvc` (the firewall) must run. Automatic services that stop cleanly by design are not reported.
- **security:** Defender is checked only when it is the active antivirus (`win_check_security_av: auto`). With another product, list its services instead.
- **certs:** warning 30 days before expiry, critical 7 days before. Old expired certificates that nothing uses are listed in the report, not raised as findings.
- **patching:** warning when the last update is 35 days old, critical at 60 days.

If something goes wrong

What you see	What to do
<code>winrm or requests is not installed / connection test: no winrm</code>	the EE has no pywinrm: step 2
<code>CERTIFICATE_VERIFY_FAILED</code>	<code>ansible_winrm_server_cert_validation: ignore</code> , or the CA bundle (step 4)
<code>the specified credentials were rejected by the server</code>	username format (<code>DOMAIN\user</code> for NTLM), password, or the account is locked
<code>Connection refused / timed out on 5986</code>	the firewall, or no HTTPS listener (step 1)
<code>kerberos: ... kinit or Server not found in Kerberos database</code>	no Kerberos in the EE, or a short host name: use NTLM, or full names
<code>local administrator: NO</code>	add the account to the local Administrators group
<code>the <name> check could not run: ...</code>	the text after it is PowerShell's error. The other checks still ran
<code>skipping: no hosts matched</code>	the template's inventory is not <code>Windows servers</code> , or the Limit matches no server there
<code>... is not digitally signed or running scripts is disabled on this system</code>	a GPO sets the PowerShell execution policy for the machine (for example <code>AllSigned</code>), which overrides what Ansible asks for. Set it to <code>RemoteSigned</code> by GPO for these servers, or sign the scripts with your code-signing certificate

Not in this release (next)

- ACT in the Windows jobs: `use_act` on the Windows health check and troubleshoot (explain, propose fixes for approval), like on Linux. ACT is already on the servers (above).
- Service watch for Windows services and IIS application pools.
- Checks for domain controllers, IIS and SQL Server.

Appendix: credential types to paste into AAP

Each block below is one file in `aap/credential_types/`. In AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**. Paste the part under `inputs:` into **Input configuration** and the part under `injectors:` into **Injector configuration** (without those two top-level lines). Copy from the repository file where you can; PDF copy can change spaces.

ServiceNow API (`servicenow_api.yml`)

Used by playbooks/`servicenow_tickets.yml` and playbooks/`servicenow_health.yml` (REST Table API). The same variable names as the certified `servicenow.itsm` collection, so this credential also works if you move to that collection later.

YAML

```
inputs:
  fields:
    - id: instance
      type: string
      label: Instance URL
      help_text: e.g. https://servicenow.example.mil
    - id: username
      type: string
      label: API user name
    - id: password
      type: string
      label: API password
      secret: true
  required: [instance, username, password]
injectors:
  env:
    SN_HOST: "{{ instance }}"
    SN_USERNAME: "{{ username }}"
    SN_PASSWORD: "{{ password }}"
```

MariaDB monitor (`mariadb_monitor.yml`)

For the mariadb health check. Needed when MariaDB runs in a container (the official images give root a password); optional on a host install, where the check can log in as root through the local socket. The password reaches the client only through its environment (`MYSQL_PWD`), never the command line. The account needs no data access at all (`docs/RUNBOOKS.md` shows the `GRANT`).

YAML

```
inputs:
  fields:
    - id: username
      type: string
      label: Monitor account
    - id: password
      type: string
      label: Password
      secret: true
  required: [username, password]
injectors:
  env:
    MARIADB_MONITOR_USER: "{{ username }}"
    MARIADB_MONITOR_PASSWORD: "{{ password }}"
```

Keystore password (keystore_password.yml)

Optional, for the certs check on a PKCS12 keystore (JKS keystores list without a password). In the inventory, name the variable, never the password: check_certs_keystores: [{path: /path/to/keystore, password_env: KEYSTORE_PASSWORD}]

```
YAML
inputs:
  fields:
    - id: password
      type: string
      label: Keystore password
      secret: true
      required: [password]
  injectors:
    env:
      KEYSTORE_PASSWORD: "{{ password }}"
```

STIG Manager API (stigman_api.yml)

Optional, for the STIG Manager cross-check in playbooks/poam_status.yml. The client is created by your Keycloak realm owner (client-credentials grant, scope stig-manager:collection:read, and a grant in the collections to read) - this repository never changes the realm.

```
YAML
inputs:
  fields:
    - id: token_url
      type: string
      label: Token URL
      help_text: https://keycloak.example.mil/realms/<realm>/protocol/openid-connect/token
    - id: client_id
      type: string
      label: Client ID
    - id: client_secret
      type: string
      label: Client secret
      secret: true
      required: [token_url, client_id, client_secret]
  injectors:
    env:
      STIGMAN_TOKEN_URL: "{{ token_url }}"
      STIGMAN_CLIENT_ID: "{{ client_id }}"
      STIGMAN_CLIENT_SECRET: "{{ client_secret }}"
```

ACT model key (act_model_key.yml)

```
YAML
inputs:
  fields:
    - id: genai_key
      type: string
      label: GenAI API key
      secret: true
    - id: asksage_key
      type: string
      label: AskSage API key
      secret: true
injectors:
  env:
    GENAI_KEY: "{{ genai_key }}"
    ASKSAGE_API_KEY: "{{ asksage_key }}"
```

ACT GenAI beta key (act_genai_beta_key.yml)

Only for ACT with site_act_provider=genai-beta (the GenAI.mil beta proxy). A separate type, so an "ACT model key" type that is already in use does not have to change (AAP refuses to change the fields of a credential type that credentials use). Attach it next to "ACT model key".

```
YAML
inputs:
  fields:
    - id: genai_beta_key
      type: string
      label: GenAI beta API key
      secret: true
      required: [genai_beta_key]
injectors:
  env:
    GENAI_BETA_KEY: "{{ genai_beta_key }}"
```